

Hafta 10 – Sertifikalar ve Kriptografik Yöntemler

Tarih	20.11.2026
Öğrenme çıktıları	ÖÇ.2, 4
Süre	3 saat
Ön bilgi	Hafta 3'ten AES-GCM, HMAC, anahtar türetme ve TLS'in ne yaptığı; terminalde <code>openssl</code> komutunu çalıştırabilmek
Uygulamalar	<code>code/week-10</code> – 11 demo (2 kabuk/OpenSSL, 9 Python); OpenSSL 1.1.1+ ya da 3.x yeterli; her demo klasöründe <code>sh demo.sh / python <demo>.py</code>



Bu haftanın çalışan demoları (11 demo)

`code/week-10/01-pki-chain` – OpenSSL ile kök → ara → sunucu zinciri; ara sertifika olmadan `verify` hatası, ekleyince OK; süresi dolmuş/yanlış kök/CA olmayan imzalayıcı/yanlış ad da reddedilir. · `code/week-10/02-signature-verification` – ECDSA (P-256) imza/doğrulama: doğru imza kabul, kurcalanan reddedilir; sürüm geri alma ve yanlış anahtar tuzakları. · `code/week-10/03-block-cipher-modes` ... `11-hsm-key-use` – Python: blok kipleri (ECB/CBC/CTR/GCM benzeri), **PKCS#7** dolgu, HMAC iç/dış özet, şifrele-sonra-MAC sırası, oyuncak RSA, oyuncak Diffie–Hellman + araya girme, oyuncak ECDH, CRL/OCSP, **HSM/PKCS#11** anahtar kullanımı.

Çalıştırma: `sh demo.sh` (kabuk demoları) ya da `python <demo>.py` (Python demoları); ikisi de yalnızca kendi geçici klasörlerini kullanır.

Demoyu kendiniz çalıştırın — adım adım (kopyala-yapıştır)

İlk iki demo **OpenSSL** ve kabuk betiği kullanır; Windows'ta **WSL** ya da **Git Bash** açın. `code` klasöründen:

```
# WSL / Linux / Git Bash
cd week-10/01-pki-chain
sh demo.sh          # kök CA -> ara CA -> sunucu sertifikası üretir ve zinciri dokuz farklı
senaryoda doğrular

cd ../02-signature-verification
sh demo.sh          # ECDSA (P-256) ile imzala -> doğrula -> tek bayt kurcala -> doğrulama
reddetsin
```

Beklenen çıktı: Birinci demo üç halkalı bir **zincir** kurar; `openssl verify` zinciri **geçerli** bulur, **ara sertifika çıkarılınca doğrulama başarısız** olur (bu haftanın "eksik ara sertifika" kuralı); ayrıca süresi dolmuş, yanlış kök, `CA:FALSE` imzalayıcı ve yanlış ad senaryolarının hepsi ayrı ayrı reddedilir. İkinci demo bir dosyayı imzalayıp doğrular (**OK**), sonra tek bir baytı değiştirir ve doğrulama **reddeder** — imza bütünlüğü böyle yakalar.

Kalan dokuz demo saf Python'dur, bağımlılık gerekmez:

```
cd ../03-block-cipher-modes
python block_modes.py    # ECB/CBC/CTR/bir GCM benzeri kip: zincirleme, IV, tek bit çevirmenin
etkisi
```

Aynı kalıp 04-pkcs7-padding (python pkcs7.py), 05-hmac-construction (python hmac_construction.py), 06-encrypt-then-mac (python ordering.py), 07-rsa-toy (python rsa_toy.py), 08-diffie-hellman (python dh_toy.py), 09-ecdh-toy (python ecdh_toy.py), 10-revocation (python revocation.py) ve 11-hsm-key-use (python hsm_sim.py) için de geçerlidir — her klasörün kendi `README.md`'si "Try it yourself" bölümünde birer öneri verir.

Bu haftanın sonunda şunları yapabileceksiniz

1. Bir uygulama için **algoritma, anahtar uzunluğu ve kip** seçmek ve seçimi güncel standartlara (NIST SP 800-57, SP 800-131A) dayandırmak.
2. Blok şifre **kiplerini** (ECB, CBC, CTR, GCM) ve **dolguyu** karşılaştırmak; dolgu kâhini (padding oracle) saldırısının neden mümkün olduğunu açıklamak.
3. **HMAC**'i doğru kullanmak; şifreleme ile MAC'i doğru sırada birleştirmek ve **yeniden oynatmaya** karşı korumak.
4. **RSA** (OAEP, PSS) ve **eliptik eğri** (Ed25519, X25519) algoritmalarıyla şifreleme, imzalama ve anahtar değişimi yapmak; imzalı Diffie–Hellman'ın neden gerektiğini göstermek.
5. **PKI** bileşenlerini (CA, RA, zincir, güven deposu) açıklamak; OpenSSL ile **kök** → **ara** → **uç** sertifika zinciri kurmak ve doğrulamak.
6. Sertifika **iptalini** (CRL, OCSP, OCSP yanıtını iliştiirme (OCSP stapling)) açıklamak ve anahtarları bir yazılım HSM'inde (**SoftHSM**, **PKCS#11**) saklamanın mantığını bilmek.

Ders akışı (öğretim üyesi için zaman planı)

Zaman	Bölüm	Ne yapılıyor
0:00–0:15	1	Kriptografinin haritası; algoritma ve anahtar uzunluğu seçimi
0:15–0:50	2–3	Blok şifre kipleri ve dolgu; MAC, HMAC, önce şifrele, sonra MAC (encrypt-then-MAC), yeniden oynatma
0:50–1:00	Ara	
1:00–1:30	4–5	RSA (OAEP, PSS) ve eliptik eğriler; dijital imza oluşturma ve doğrulama (OpenSSL)
1:30–1:50	6	Anahtar değişimi: Diffie–Hellman, araya girme, imzalı DH
1:50–2:00	Ara	
2:00–2:40	7–9	PKI bileşenleri; X.509 yapısı; OpenSSL ile sertifika zinciri kurmak — sınıf etkinliği
2:40–2:55	10–11	İptal (CRL, OCSP); anahtar saklama (PKCS#11 , SoftHSM, HSM)
2:55–3:00	12+	Kuantum sonrası kriptografiye bakış, proje adımı, kendini sına

Laboratuvarı önceden hazırlayın

Bu haftanın komutları **OpenSSL 3** ister (Windows'ta Git for Windows ile gelen `openssl`, WSL/Linux'ta paket yöneticisi). Bütün dosyaları boş bir çalışma klasöründe üretin; üretilen anahtarlar **yalnız ders içindir**, hiçbir gerçek sistemde kullanılmaz. SoftHSM bölümü isteğe bağlıdır.

```
openssl version
mkdir C:\Dersler\h10; cd C:\Dersler\h10

sudo apt install -y openssl softhsm2 openc
openssl version
mkdir -p ~/h10 && cd ~/h10
```

Etik kural — bu derste her hafta geçerli

Bu haftanın bütün anahtarları ve sertifikaları kendi ürettiğimiz, **sentetik** ve yalnız laboratuvar içindir. Kendi ürettiğiniz bir kök sertifikayı başkasının bilgisayarının güven deposuna eklemek ya da bir sertifika otoritesinin adını taşıyan sahte sertifika üretmek kabul edilemez. Araya girme örnekleri yalnız kendi bilgisayarınızda, `localhost` üzerinde yapılır.

0. Temel kavramlar (sıfırdan)

Bu bölüm haftaya hazırlık içindir. Önce bu haftanın dayandığı önceki konuları kısaca hatırlatır; sonra bu haftanın kavramlarını birer cümleyle tanımlayıp her birini ayrıntılı anlatıldığı bölüme bağlar.

Önceki haftalardan gelenler

- **Simetrik ve asimetrik şifreleme** — simetrik şifreleme iki tarafın paylaştığı tek bir anahtarla hızlı çalışır; asimetrik şifreleme açık ve özel anahtar çiftiyle anahtar dağıtımını kolaylaştırır ama yavaştır ([Hafta 3, §2.2](#)). Bu hafta asimetrik tarafın matematiğini (RSA, ECC — §4) ve simetrik tarafın doğru kip/dolgu seçimini (§2) derinleştiriyoruz.
- **AEAD** — kimlik doğrulamalı şifreleme, veriyi tek bir çağrıda hem şifreleyip hem de değişmediğini kanıtlayan bir etiket üreterek gizlilik ve bütünlüğü birlikte sağlar ([Hafta 3, §2.1](#)). Bu hafta AEAD'in içindeki kip (GCM) ile ondan uzak durulması gereken alternatiflerin (ECB, CBC) neden farklı güvenlik verdiğini gösteriyoruz (§2).
- **Özet (hash), MAC ve dijital imza** — özet veriden hesaplanan tek yönlü bir parmak izidir; MAC paylaşılan bir anahtarla, dijital imza ise açık/özel anahtar çiftiyle mesajın değişmediğini kanıtlar ([Hafta 3, §2.4](#)). Bu hafta HMAC'in iç yapısını ve şifrelemeyle doğru birleştirme sırasını (§3), imza doğrulamasının pratikteki tuzaklarını (§5) işliyoruz.
- **İleri gizlilik (forward secrecy)** — oturum için türetilen anahtarın kullanımdan sonra silinmesiyle, bugünkü anahtar sızsa bile geçmiş oturumların çözülememesidir ([Hafta 3, §7.1](#)). Bu hafta bunun temelindeki Diffie–Hellman anahtar değişimini ve kimlik doğrulamasını sıfırdan kuruyoruz (§6).
- **TLS sertifika doğrulama** — istemcinin bir sunucu sertifikasını zincir, geçerlilik tarihi, kullanım amacı ve ad açısından denetlemesidir ([Hafta 3, §9.2](#)). Bu hafta bu doğrulamanın dayandığı PKI, X.509 ve iptal (CRL/OCSP) mekanizmalarını baştan kuruyoruz (§7–10).

Bu haftanın kavram haritası

Kavram	Bir cümlede	Ayrıntısı
Blok şifre kipleri ve dolgu	Bir blok şifre sabit boyutlu bir bloğu şifreler; bu blokları uzun bir mesaja uygulama yöntemine kip denir ve mesaj blok boyutunun katı değilse dolgu ile tamamlanır; kip ve dolgu seçimi algoritmanın kendisi kadar güvenliği belirler.	§2
MAC ve HMAC	MAC, gizli bir anahtar ve bir mesajdan, mesajın değişmediğini kanıtlayan kısa bir etiket üreten fonksiyondur; HMAC bunu bir özet fonksiyonundan iç içe iki çağrıyla türetmenin standart yoludur.	§3
Asimetrik kriptografi: RSA ve ECC	RSA büyük tam sayıların çarpanlarına ayrılmasının, eliptik eğri kriptografisi (ECC) ise bir eğri üzerindeki ayrık logaritma probleminin zorluğuna dayanır; ECC aynı güvenliği çok daha kısa anahtarlar verir.	§4
Dijital imza	Bir mesajın belirli bir özel anahtarın sahibi tarafından oluşturulduğunu ve değiştirilmediğini, herkesin açık anahtarla doğrulayabildiği; MAC'ten farklı olarak inkâr edilemezlik de sağlayan bir mekanizmadır.	§5
Diffie–Hellman (DH)	İki tarafın önceden hiçbir sır paylaşmadan, güvensiz bir kanal üzerinden ortak bir sırda anlaşmasını sağlayan bir anahtar değişimi yöntemidir; kimlik doğrulanmazsa araya girme saldırısına açıktır.	§6
PKI ve CA	Açık anahtar altyapısı (PKI), bir açık anahtarın gerçekten iddia edilen sahibine ait olduğunu sertifika otoriteleri (CA) aracılığıyla doğrulayan kurallar, roller ve belgeler bütünüdür.	§7
Sertifika ve X.509	X.509, bir açık anahtarı bir kimliğe bağlayan ve CA'nın imzasını taşıyan sertifikaların standart biçimidir; içinde konu, açık anahtar, geçerlilik tarihleri ve uzantılar (SAN, anahtar kullanımı) bulunur.	§8
CRL ve OCSP	CRL, iptal edilmiş sertifikaların CA tarafından imzalanmış listesidir; OCSP ise bir sertifikanın iptal durumunu anlık sorgulamayı sağlayan protokoldür.	§10
HSM, PKCS#11, SoftHSM	HSM, kurcalamaya dayanıklı bir donanımda anahtar üretip işleyen, anahtarı asla dışarı vermeyen bir modüldür; PKCS#11 bu modüllerle konuşmanın standart arayüzüdür ve SoftHSM bunun test amaçlı yazılım benzetimidir.	§11
Kuantum sonrası kriptografi (PQC)	Yeterince büyük bir kuantum bilgisayarın RSA'nın ve eliptik eğrinin dayandığı matematiksel problemleri verimli çözebileceği varsayımına karşı, kafes gibi farklı matematiksel problemlere dayanan yeni algoritmalar geliştirir.	§12

Kavramlar birbirine nasıl bağlanır?

Yukarıdaki terimler rastgele bir liste değildir; her biri bir öncekinin üstüne oturur. Aşağıdaki tablo "bunu anlamak için önce şunu bilmem gerekir" ilişkisini gösterir — bir terimi unutursanız tablodaki "önkoşul" sütununa geri dönün.

Terim	Önkoşulu	Hangi bölümde derinleşir?
Blok şifre / kip	Simetrik kriptografi	Bölüm 2
Dolgu	Blok şifre / kip	Bölüm 2
AEAD	Blok şifre, MAC	Bölüm 2–3
MAC / HMAC	Özet (hash), simetrik anahtar	Bölüm 3
Dijital imza	Asimetrik kriptografi, özet	Bölüm 4–5
RSA / ECC	Asimetrik kriptografi	Bölüm 4
Diffie–Hellman	Asimetrik kriptografi, imza	Bölüm 6
PKI / CA	Dijital imza, güven	Bölüm 7
Sertifika / X.509	PKI, imza, özet	Bölüm 8–9
CRL / OCSP	Sertifika, CA	Bölüm 10
HSM / PKCS#11	Özel anahtar kavramı	Bölüm 11
PQC	RSA/ECC'nin dayandığı matematik problemleri	Bölüm 12

Bu tablonun okunuş yönü önemlidir: **satır başındaki terimi anlamak için önce "önkoşulu" sütunundaki terimi bilmelisiniz.** Örneğin "sertifika iptali" (CRL/OCSP) bölümünü okumadan önce "sertifika" ve "CA" kavramlarının oturmuş olması gerekir; aksi hâlde "CA neden iptal listesini imzalıyor?" sorusu havada kalır.



Bu haftayı nasıl okumalı?

Bölümler **sırayla** birbirinin üstüne inşa edilir: 1 → 2 → 3 (simetrik araçlar), 4 → 5 → 6 (asimetrik araçlar), 7 → 8 → 9 → 10 (bunları PKI'ya oturma), 11 (anahtarı saklama), 12 (geleceğe bakış). Bir bölümü atlayıp ileri giderseniz, sonraki bölümdeki "3. haftadaki X'i hatırlayın" ya da "bir önceki bölümdeki Y" gibi cümleler anlamsız kalır. Zor gelen bir bölümü ikinci kez okuyup ilerleyin.

1. Kriptografinin haritası ve algoritma seçimi

Dokuzuncu haftayı bir uyarıyla bitirmiştik: kod gizleme bir anahtarı **saklamaz**, yalnız çevresini zorlaştırır ([Hafta 9, §3](#)). Bu hafta tam da bunun çözümüne geçiyoruz — anahtarları gizlemek yerine, doğru algoritma seçimi, doğru protokol ve gerektiğinde donanım destekli saklama (PKI, HSM) ile **gerçekten** koruyoruz. Üçüncü haftada veriyi aktarımda, beklemede ve kullanımda korumak için kriptografiyi bir **araç** olarak kullanmıştık ([Hafta 3, §1](#)); bu hafta aynı araçların içine bakıyoruz: hangi algoritma hangi işi yapar, neden bazıları terk edildi, nasıl seçilir ve bir sertifika otoritesi (CA) bu yapıya nasıl oturur. Kitabın 4–8. ve 10–11. bölümleri, 55'ten fazla tarifle bu konuları işler; 2003'ten beri algoritmaların çoğu değiştiği için her birini güncel karşılığıyla veriyoruz.

Simetrik ve asimetrik: hangisi ne için?

pratikte HİBRİT: asimetrikle anahtar kur, simetrikle veri şifrele

SİMETRİK (AES)

tek anahtar
ÇOK HIZLI

sorun: anahtarı nasıl paylaşırsın?

ASİMETRİK (RSA/EC)

açık + gizli anahtar çifti
YAVAŞ

anahtar dağıtımı ve İMZA çözülür

TLS tam olarak bunu yapar: ECDHE ile anahtar, AES-GCM ile veri.



Kısa tarihçe: anahtarlar, sertifikalar ve PKI

- **1976–77** — Diffie–Hellman ve **RSA**: açık anahtar, "tanımadığın biriyle güvenli konuşma" sorununu çözer.
- **1988** — **X.509** sertifika biçimi standartlaşır; kimliği bir **CA imzası** taşır.
- **1995** — ilk ticari CA'lar (VeriSign) ve **PKI** yaygınlaşır; ardından iptal mekanizmaları **CRL** ve **OCSP** gelir.
- **2014** — **Heartbleed** ve POODLE, TLS ekosistemini sıkılaştırır; **2015 Let's Encrypt** ücretsiz sertifikayla HTTPS'i yaygınlaştırır; **2018** TLS 1.3.
- **2022–2024** — NIST **kuantum sonrası** algoritmaları seçer (Kyber/ML-KEM, Dilithium/ML-DSA).

Bu haftanın tüm kuralları (doğru kip, doğru dolgu, zincir doğrulama, iptal) bu çizginin acı derslerinden çıkmıştır.

Beş temel iş, beş araç ailesi

İş	Araç	Güncel seçim	Terk edilen
Gizlilik (simetrik)	Blok ve akış şifreleri	AES-256-GCM, ChaCha20-Poly1305	DES, 3DES, RC4, Blowfish
Bütünlük + kimlik (simetrik)	MAC	HMAC-SHA-256, CMAC, GCM'in etiketi	CBC-MAC (değişken uzunlukta), MD5 tabanlı MAC'ler
Parmak izi	Özet fonksiyonu	SHA-256, SHA-384, SHA-3	MD5, SHA-1
Anahtar değişimi	Diffie–Hellman, RSA ile anahtar taşıma	X25519, ECDHE (P-256)	1024 bit DH, RSA ile anahtar taşıma (TLS 1.3'te kaldırıldı)
İmza (asimetrik)	Dijital imza	Ed25519, ECDSA P-256, RSA-PSS ≥ 3072	DSA, RSA PKCS#1 v1.5 imzası (yeni tasarımlarda)

Anahtar uzunluğu: güvenlik düzeyi

Farklı algoritmaların anahtar uzunlukları doğrudan karşılaştırılmaz. NIST SP 800-57, bunları **eşdeğer güvenlik düzeyinde** (bit cinsinden) karşılaştırır:

Güvenlik düzeyi	Simetrik	Özet (çakışma direnci)	RSA / DH	Eliptik eğri
112 bit	3DES (yalnız eski sistemler)	SHA-224	2048	224
128 bit	AES-128	SHA-256	3072	256 (P-256, Ed25519/X25519)
192 bit	AES-192	SHA-384	7680	384
256 bit	AES-256	SHA-512	15360	512

Bugün yeni bir tasarım için alt sınır **128 bit güvenlik düzeyidir**. Bu, RSA'da 3072 bit, eliptik eğride 256 bit demektir; 2048 bitlik RSA, NIST'in geçiş planında 2030'dan sonra yeni kullanım için yeterli sayılmaz. Tablonun en önemli dersi, eliptik eğrilerin aynı güvenliği **çok daha kısa** anahtarlarla sağlamasıdır; mobil ve gömülü sistemlerde tercih edilmelerinin nedeni budur.



Kitaptaki karşılığı

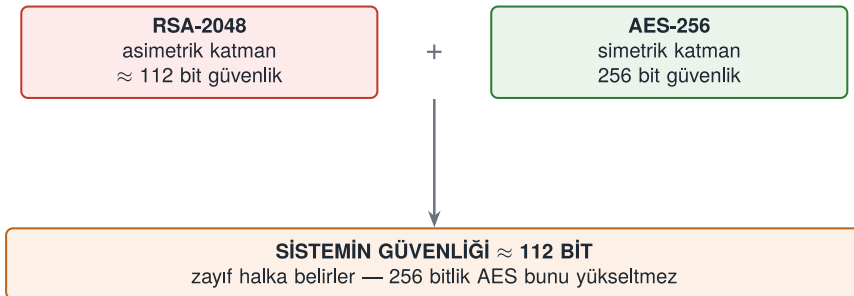
Kitap Tarif 5.2 ve 5.3'te algoritma ve anahtar uzunluğu seçimini, 6.3 ve 6.4'te özet ve MAC seçimini, 7.2 ve 7.3'te açık anahtar algoritması ve uzunluğu seçimini işler. Önerdiği ilke bugün de geçerlidir: **iyi incelenmiş, standart bir algoritma seçin ve onu yüksek düzeyli, hata yapmaya dirençli bir API ile kullanın** (Tarif 5.16). Değişen yalnız algoritmaların kendisidir.

En zayıf halka kuralı

Bir sistem birden çok kriptografik katman kullanır: anahtar değişimi bir algoritmayla, veri şifreleme başka biriyle yapılır. Sistemin güvenlik düzeyi bu katmanların **en düşüğüne** eşittir — en güçlüsüne değil. RSA-2048 ile anahtar taşıyıp veriyi AES-256 ile şifrelerseniz, saldırgan 256 bitlik AES'i kırmaya çalışmaz; ≈ 112 bitlik RSA katmanını hedef alır. Bu yüzden katmanları tek tek değil, **birlikte** ve en düşük seviyeye göre değerlendirin.

En zayıf halka kuralı

sistemin gücü, en güçlü parçası kadar değildir



Katmanları ayrı ayrı değil, en düşük güvenlik seviyesine göre değerlendirin.

Bir kural: kendi kriptonuzu yazmayın, kendi protokolünüzü de

Algoritmaların hepsi doğru olsa bile, onları birleştirmek (hangi sırayla şifreleyip MAC'leyeceğinizi, nonce'u nereden alacağınızı, anahtarları nasıl ayıracağınızı seçmek) ayrı bir uzmanlık ister. Bu haftanın bütün örnekleri, sahadaki hataların neredeyse hepsinin **algoritmada değil, birleştirmede** olduğunu gösterecek. Mümkün olan her yerde hazır, kanıtlanmış yapıları (TLS 1.3, AEAD, HPKE, Noise) kullanın.

Sayısal örnek: "128 bit" ile "112 bit" arasındaki fark ne kadar büyük?

Tablodaki bit sayıları soyut kalabilir; somutlaştıralım. Bir güvenlik düzeyi "n bit" demek, saldırganın ortalama 2^{n-1} işlem denemesi gerektiği anlamına gelir (üst sınır olarak 2^n kullanılır). RSA-2048 (112 bit) ile AES-128 (128 bit) arasındaki fark, sanıldığından çok daha büyüktür:

Elle hesap (üstel büyüme)

$2^{112} \approx 5,19 \times 10^{33}$ (RSA-2048'in eşdeğer zorluğu)
 $2^{128} \approx 3,40 \times 10^{38}$ (AES-128'in anahtar uzayı)
 $2^{128} / 2^{112} = 2^{16} = 65.536 \rightarrow$ AES-128'i kırmak, RSA-2048'i "eşdeğer" zorlukta kırmaktan 65.536 KAT daha zor

Bunu zamana çevirelim. Bugünkü en hızlı süper bilgisayarların saniyede yapabileceği işlem sayısını **iyimser** bir üst sınırla 10^{18} (kentilyon; exaölçek süper bilgisayar) olarak alalım — gerçek saldırılarda bu sayı çok daha düşüktür, çünkü şifre kırma tek bir aritmetik işlem değil, çok daha pahalı bir işlemdir. Yine de üst sınırla hesaplayalım:

2^{112} işlemi 10^{18} işlem/saniye ile ne kadar sürer?

2^{112} işlem $\div 10^{18}$ işlem/saniye $\approx 1,6 \times 10^8$ yıl (≈ 165 milyon yıl)
 Karşılaştırma: yazılı insanlık tarihi ≈ 5.000 yıl $\rightarrow 165$ milyon yıl, bunun ~ 33.000 katı
 2^{128} işlem $\div 10^{18}$ işlem/saniye $\approx 1,08 \times 10^{13}$ yıl
 Karşılaştırma: evrenin yaşı $\approx 1,38 \times 10^{10}$ yıl $\rightarrow 2^{128}$, evrenin yaşının ~ 780 katı kadar sürer

Çıkarılacak ders iki yönlüdür. Birincisi, **112 bit bile bugün için pratikte kaba kuvvetle kırılmaz**; RSA-2048'in 2030 sonrası için "yetersiz" sayılmasının nedeni kaba kuvvet değil, (a) donanımın zamanla ucuzlaması, (b) uzun süre saklanması gereken verilerin gelecekteki saldırılara karşı **pay bırakılarak** korunması gerekliliği ve © kuantum bilgisayarların RSA'yı kaba kuvvetten çok daha hızlı kırabilecek olmasıdır (bkz. Bölüm 12). İkincisi, 112 bit ile 128 bit arasındaki görünüşte küçük fark (16 bit), saldırganın işini **65 binde bir**e indirger — bit sayılarını doğrusal değil, **üstel** okumalısınız.



Sık yapılan hata: 'AES kullanıyoruz' demek yeterli sanmak

Bir algoritma envanterinde yalnız algoritmanın adını yazmak (AES) hiçbir şey söylemez. AES-128-ECB ile AES-256-GCM aynı ailenin üyesidir ama güvenlik açısından yerden göğe farklıdır: biri (ECB) desen sızdırır (Bölüm 2), diğeri (GCM) bütünlüğü de sağlar. Bir algoritma envanterinde **algoritma + kip + anahtar uzunluğu + dayandığı standart** birlikte yazılmalıdır (kitabın Tarif 5.16'sı ve bu haftanın 13. bölümündeki "S8" şablonu).



Kural

Bir kriptografik seçimi yalnız algoritma adıyla değil, (**algoritma, anahtar uzunluğu, kip, kütüphane, standart**) beşlisiyle belgeleyin. "AES kullanıyoruz" bir cevap değildir; "AES-256-GCM, OpenSSL EVP, NIST SP 800-38D" bir cevaptır.

Üçüncü haftada bu haritanın yalnız "hangi aracı ne zaman kullanırım" kısmını gördük (AEAD, hibrit şema, rastgele sayı). Bu hafta aynı haritanın **neden** böyle çizildiğini — hangi matematiksel problem hangi güvenliği veriyor, hangi algoritma neden terk edildi — görüyoruz. Sıradaki iki bölüm simetrik araçların (kip, dolgu, MAC) içine; 4–6. bölümler asimetrik araçların (RSA, ECC, DH) içine; 7–10. bölümler bunların PKI'da nasıl bir araya geldiğine bakacak.

2. Blok şifre kipleri ve dolgu (Tarif 5.4, 5.11)

AES bir **blok şifredir**: tam olarak 16 baytlık bir bloğu, 16 baytlık başka bir bloğa çevirir. Daha uzun bir mesajı şifrelemek için blokları birbirine bağlayan bir **kip** (mode of operation) gerekir. Kip seçimi, algoritma seçimi kadar önemlidir; **3. haftadaki ECB penguenini** hatırlayın.

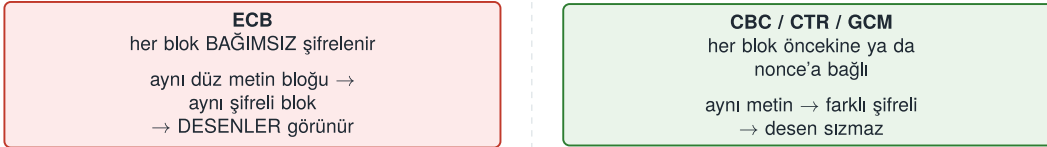
Nonce tekrarı neden felaket?

gizlilik VE bütünlük birlikte çöker



ECB neden asla kullanılmaz?

ünlü "ECB pengueni" bu yüzden görünür kalır



Kip seçimi şifrelemenin kendisi kadar önemlidir; varsayılan tercih AEAD (GCM).

Kip	Nasıl çalışır?	Gizlilik	Bütünlük	Paralel	Dolgu	Değerlendirme
ECB	Her blok bağımsız şifrelenir	X (desen sızar)	X	✓	Gerekir	Kullanmayın
CBC	Her blok önceki şifreli blokla XORlanır	✓ (IV tahmin edilemezse)	X	Yalnız çözme	Gerekir	Ayrı MAC ile; dolgu kâhini dikkat
CTR	Sayaç şifrelenip anahtar akışı üretilir	✓ (nonce benzersizse)	X	✓	Gerekmez	Ayrı MAC ile
GCM	CTR + çarpım tabanlı kimlik doğrulama	✓	✓	✓	Gerekmez	Önerilen (AEAD)
ChaCha20-Poly1305	Akış şifresi + Poly1305 MAC	✓	✓	✓	Gerekmez	Önerilen ; AES donanım desteği olmayan cihazlarda hızlı

Dolgu ve dolgu kâhini

ECB ve CBC, mesaj uzunluğunun blok boyutunun katı olmasını ister; eksik kısım **dolgu** ile tamamlanır. En yaygın dolgu **PKCS#7**'dir: eksik bayt sayısı n ise, n adet n değerli bayt eklenir (ör. 3 bayt eksikse `03 03 03`). Çözen taraf son baytı okuyup o kadar baytı atar; ama önce dolgunun **geçerli** olup olmadığını denetler.

Sorun burada başlar: çözme hatası "dolgu geçersiz" ve "MAC geçersiz" olarak **ayırt edilebiliyorsa** (farklı hata iletisi, farklı yanıt süresi), saldırgan şifreli metnin baytlarını değiştirip sunucunun tepkisini gözleyerek mesajı bayt bayt çözebilir. Buna **dolgu kâhini** (padding oracle) saldırısı denir; Serge Vaudenay 2002'de tanımladı, sonraki yıllarda web çerçevelerinde ve TLS'te (POODLE, Lucky Thirteen) tekrar tekrar karşımıza çıktı.

Savunmalar:

1. **AEAD kullanın** (GCM, ChaCha20-Poly1305): dolgu yoktur, etiket tutmazsa hiçbir şey çözülmez.
2. CBC zorunluysa **önce şifrele, sonra MAC'le** (bir sonraki bölüm) ve MAC'i **dolguyu denetlemeden önce** doğrulayın.
3. Bütün çözme hatalarını **tek ve aynı** hata olarak, aynı sürede döndürün.

**Sahada nasıl uygulanır?**

Sertifikasyondan geçmiş bir mobil ödeme kütüphanesinin algoritma envanterinde AES'in CBC ve CTR kiplerinde, ayrı bir SHA-256 tabanlı MAC ile kullanıldığı görülür; ödeme şemalarının kriptogram hesabı için ise ISO/IEC 9797-1'e dayanan, DES tabanlı bir MAC algoritması (şemaların belirlediği dolgu yöntemleriyle) kullanılır. Bu seçimler, uyulması gereken şema belirtilmelerinden gelir. Şemanın belirlemediği iç iletişimde ise bugün AEAD tercih edilir ([3. haftadaki eleştirel okuma](#)).

İşlenmiş örnek: PKCS#7 dolgusunu elle izlemek

"3 bayt eksikse 03 03 03 eklenir" kuralını soyut bırakmayalım; gerçek bir OpenSSL çalıştırmasıyla adım adım izleyelim. 16 baytlık sabit bir anahtar ve sıfırdan artan bir IV (yalnız bu örnek için; gerçek sistemde IV her mesajda rastgele üretilir) ile 13 baytlık "MERHABA DUNYA" metnini AES-128-CBC ile şifreleyelim:

PKCS#7 dolgusunu elle izlemek (gerçek çıktı)

```
K=00112233445566778899aabbccddeeff # 16 bayt (32 hex karakter) – yalnız gösterim için sabit
IV=000102030405060708090a0b0c0d0e0f # 16 bayt – yalnız gösterim için sabit
printf 'MERHABA DUNYA' | openssl enc -aes-128-cbc -K "$K" -iv "$IV" -out cikti.bin
xxd -p cikti.bin
```

Çıktı (gerçekten üretilen değer):

```
4b354371f98acde5bed95d59a8135538
```

Girdi 13 bayttı; AES'in blok boyu 16 bayttır; çıktı da tam **16 bayt** (32 hex karakter) — demek ki 3 baytlık bir dolgu eklenmiş. Dolgunun **içeriğini** görmek için `-nopad` ile çözelim (normalde bunu hiç yapmayız; burada yalnız öğretici amaçla):

```
openssl enc -d -aes-128-cbc -K "$K" -iv "$IV" -in cikti.bin -nopad | xxd
```

```
00000000: 4d45 5248 4142 4120 4455 4e59 4103 0303  MERHABA DUNYA...
```

Son üç bayt 03 03 03 — tam olarak PKCS#7 kuralının söylediği gibi: eksik bayt sayısı ($16 - 13 = 3$) kadar, o sayı değerinde bayt eklenmiş. Normal `openssl enc -d` (dolgusuz bayrak olmadan) bu üç baytı otomatik okuyup atar ve size tam olarak "MERHABA DUNYA" döndürür — bunu da doğrulayın:

```
openssl enc -d -aes-128-cbc -K "$K" -iv "$IV" -in cikti.bin
```

```
MERHABA DUNYA
```

Dolgu kâhini saldırısı tam bu noktada devreye girer: saldırgan şifreli metnin son baytlarını değiştirip sunucuya gönderir; sunucu "dolgu geçersiz" (ör. son bayt 07 ama önceki 7 bayt 07 değil) ile "dolgu geçerli ama MAC geçersiz" hatalarını **farklı** döndürüyorsa, saldırgan bu iki durumu ayırt ederek düz metni bayt bayt yeniden kurabilir — anahtara hiç ihtiyaç duymadan.

Dolgunun eklenmesini ve üç denetimin (uzunluk, son bayt aralığı, her dolgu baytının değeri) sırayla nasıl çalıştığını, hangisinin hangi bozulmada başarısız olduğunu görerek izleyin:

data = `s e c r e t M e s s a g e 0 1`

1. Girdi, 15 bayt: "SecretMessage01". Blok boyu = 8.

data = `s e c r e t M e s s a g e 0 1 1`

2. pad(data) — 1 bayt eklenir, her biri değeri 1.

data = `s e c r e t M e s s a g e 0 1 1`

3. unpad — tüm kontroller geçti: orijinal 15 bayt geri verildi.

Uyar: dolgu doğru, Zor: son bayt bozuldu ve Uç durum: uzunluk artık blok katı değil / Uç durum: bir dolgu baytı bozuldu örneklerinin hepsini deneyip hangi kontrolün nerede durduğunu karşılaştırın.

İşlenmiş örnek: ECB'nin deseni neden sızdırdığını görmek

ECB'de her 16 baytlık blok **bağımsız** şifrelenir; aynı düz metin bloğu her zaman aynı şifreli metin bloğunu üretir. 48 bayt boyunca tekrar eden bir düz metni ("AAAAAAAAAAAAAAAA" üç kez) AES-128-ECB ile şifreleyelim:

ECB'de tekrar eden blok = tekrar eden şifreli metin (gerçek çıktı)

```
K2=$(openssl rand -hex 16)
printf 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA' | head -c 48 > duz.txt # 48 bayt = 3
blok, hepsi aynı
openssl enc -aes-128-ecb -K "$K2" -in duz.txt -out cikti_ecb.bin -nopad
xxd cikti_ecb.bin
```

Örnek bir çalıştırmanın çıktısı (anahtar rastgele olduğu için siz farklı bayt değerleri göreceksiniz, ama **örüntü aynı kalacak**):

```
00000000: f5cf 4879 d8fe 9c90 1975 2de6 eb9f 59a1 ..Hy.....u-...Y.
00000010: f5cf 4879 d8fe 9c90 1975 2de6 eb9f 59a1 ..Hy.....u-...Y.
00000020: f5cf 4879 d8fe 9c90 1975 2de6 eb9f 59a1 ..Hy.....u-...Y.
```

Üç blok da **birebir aynı**. Anahtarı bilmeyen bir gözlemci bile "bu üç blok aynı düz metni içeriyor" bilgisini elde eder — ki bu genelde başlı başına bir sızıntıdır (ör. bir resmin arka planı, bir formun sabit alanları, tekrarlanan bir kayıt yapısı). AES-128-CBC ile aynı deneyi tekrarlıyorsanız (rastgele IV ile), üç blok da **farklı** çıkar; çünkü her blok bir öncekiyle XOR'lanarak zincirlenir.

Dört kipin (ECB, CBC, CTR, GCM benzeri) hepsini aynı küçük mesaj üzerinde, bayt bayt, bir kurcalamanın her kipte neye yol açtığını görerek karşılaştırın:

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

1. Düz metin, 12 bayt: "SEEDSEEDXYAB", kip: ECB.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

2. ecb_encrypt — her bayt, ÖNCEKİLERDEN BAĞIMSIZ, aynı anahtarla şifrelenir.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08

3. byte 0: toy_encrypt_block(83, key) = 08.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62
----	----

4. byte 1: toy_encrypt_block(69, key) = 62.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62	62
----	----	----

5. byte 2: toy_encrypt_block(69, key) = 62.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62	62	BB
----	----	----	----

6. byte 3: toy_encrypt_block(68, key) = BB.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62	62	BB	08
----	----	----	----	----

7. byte 4: toy_encrypt_block(83, key) = 08.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62	62	BB	08	62
----	----	----	----	----	----

8. byte 5: toy_encrypt_block(69, key) = 62.

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62	62	BB	08	62	62
----	----	----	----	----	----	----

9. byte 6: toy_encrypt_block(69, key) = 62

kip **ECB**
 plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62	62	BB	08	62	62	BB
----	----	----	----	----	----	----	----

10. byte 7: toy_encrypt_block(68, key) = BB

9. byte 8: toy_encrypt_block(88, key) = 77.

10. byte 9: toy_encrypt_block(89, key) = 1E.

kip **ECB**

plain[] = S E E D S E E D X Y A B

ct[] = 08 62 62 BB 08 62 62 BB 77

kip **ECB**

plain[] = S E E D S E E D X Y A B

ct[] = 08 62 62 BB 08 62 62 BB 77 1E

11. byte 8: toy_encrypt_block(88, key) = 77.

12. byte 9: toy_encrypt_block(89, key) = 1E.

kip **ECB**

plain[] = S E E D S E E D X Y A B

ct[] = 08 62 62 BB 08 62 62 BB 77 1E C6

kip **ECB**

plain[] = S E E D S E E D X Y A B

ct[] = 08 62 62 BB 08 62 62 BB 77 1E C6 D1

13. byte 10: toy_encrypt_block(65, key) = C6.

14. byte 11: toy_encrypt_block(66, key) = D1.

kip **ECB**

plain[] = S E E D S E E D X Y A B

ct[] = 08 62 62 BB 08 62 62 BB 77 1E C6 D1

çözülen = S E E D S E E D X Y A B

15. Çöz: her şey doğru çözüldü.

Örnek seçiciden **Uyar: ECB, tekrar eden desen sızar, Zor: CBC, şifreli metin kurcalandı** ve **Uç durum: GCM benzeri, kurcalama REDDEDİLİR** örneklerini deneyin; 🎲 ile rastgele bir mesaj/kip de üretebilirsiniz.

⚡ Sık yapılan hata: 'AES kullandık, güvenliyiz' demek

Bir kod tabanında kip belirtilmeden yapılan bir çağrı (ör. Java'da `Cipher.getInstance("AES")`), kütüphaneye göre **sessizce ECB'ye düşebilir** (Java'da "AES" kısaltması "AES/ECB/PKCS5Padding" anlamına gelir). Bu, kod incelemesinde gözden kolayca kaçan, ama üretimde gerçek veri sızdıran bir hatadır. Kural: kip **her zaman açıkça** yazılmalı ve statik analiz/derleme aşamasında ECB kullanan çağrılar reddedilmelidir.

✓ Kural

Kip adını **asla** varsayılamayın; AES-256-GCM gibi tam adıyla yazın. ECB'yi hiçbir durumda kullanmayın (dosya şifreleme, DB alanı, hatta "önemsiz" veri için bile) — deseni her zaman sızdırır. CBC zorunluysa dolguyu Bölüm 3'teki önce şifrele, sonra MAC kuralıyla koruyun.

İkinci haftadaki tehdit modellemesinde "bütünlük" bir varlık koruma hedefiydi; bu bölüm o hedefin **neden yalnız şifrelemekle sağlanamadığını** somutlaştırdı. Sıradaki bölüm, bütünlüğü doğru sağlayan aracı — MAC/HMAC'i ve doğru birleştirme sırasını — işliyor.

3. MAC, HMAC ve şifreleme ile bütünlüğün birleştirilmesi (Tarif 6.4, 6.10, 6.18, 6.21)

HMAC nasıl çalışır?

Neden HMAC, neden $H(K||m)$ değil?

kendi MAC'inizi kurmayın



Doğrulamayı sabit zamanlı karşılaştırmayla yapın (CRYPTO_memcmp).

Encrypt-then-MAC: doğru sıra

alıcı geçersiz metni hiç çözmez → dolgu kâhini kapanır



Yanlış sıra (önce çöz, sonra doğrula) dolgu hatası sızdırır ve oracle açar.

Encrypt-then-MAC: alıcı geçersiz metni hiç çözmediği için dolgu kâhini (padding oracle) için gereken "hata sızıntısı" oluşmaz. AES-GCM bu iki adımı tek işlemde yapar.

MAC (mesaj kimlik doğrulama kodu), gizli bir anahtar ve bir mesajdan kısa bir etiket üretir; anahtarı bilmeyen biri geçerli bir etiket üretemez. **HMAC**'i Bellare, Canetti ve Krawczyk 1996'da tanımladı; 1997'de RFC 2104 olarak standartlaştı. Bir özet fonksiyonundan (SHA-256) MAC üretmenin standart yoludur:

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$$

İç içe iki özet, düz $H(K \parallel m)$ yapısının **uzunluk uzatma** zayıflığını ortadan kaldırır: SHA-256 gibi Merkle–Damgård yapısındaki özetlerde $H(K \parallel m)$ biliniyorsa, anahtarı bilmeden $H(K \parallel m \parallel ek)$ hesaplanabilir. Bu yüzden **kendi MAC'inizi özetten türetmeyin**; HMAC kullanın (Tarif 6.19'un uyarısı).

İç (ipad) ve dış (opad) özetin anahtarı nasıl iki farklı sabitle karıştırdığını, anahtar bloktan uzun/kısa olduğunda ne olduğunu adım adım izleyin:

anahtar = **k e y 1**

1. Anahtar, 4 bayt: "key1". Blok boyu (oyuncak) = 8.

anahtar = **k e y 1**

2. `_normalize_key` — anahtar bloktan kısa/eşit: doğrudan sıfırla 8 bayta tamamlanır.

anahtar = **k e y 1**
 K' = 66 65 79 31 00 00 00 00
 $K' \text{ xor ipad}$ = 50 53 4f 07 36 36 36 36

3. `ipad_key = bytes(b xor 0x36 for b in K)` — anahtar 0x36 ile karıştırıldı.

anahtar = **k e y 1**
 K' = 66 65 79 31 00 00 00 00
 $K' \text{ xor ipad}$ = 50 53 4f 07 36 36 36 36
 $K' \text{ xor opad}$ = 37 39 25 60 5c 5c 5c 5c

4. `opad_key = bytes(b xor 0x5C for b in K)` — anahtar 0x5C ile karıştırıldı (FARKLI sabit).

anahtar = **k e y 1**
 K' = 66 65 79 31 00 00 00 00
 $K' \text{ xor ipad}$ = 50 53 4f 07 36 36 36 36
 $K' \text{ xor opad}$ = 37 39 25 60 5c 5c 5c 5c
 mesaj = t r a n s f e r 1 0 0 t o a c c o u n t
 iç = D618403D

5. `inner = SHA256(K'xor_ipad || message)` — İç özet: anahtar-karışımı ön ek + mesaj.

anahtar = **k e y 1**
 K' = 66 65 79 31 00 00 00 00
 $K' \text{ xor ipad}$ = 50 53 4f 07 36 36 36 36
 $K' \text{ xor opad}$ = 37 39 25 60 5c 5c 5c 5c
 mesaj = t r a n s f e r 1 0 0 t o a c c o u n t
 iç = D618403D
 dış (HMAC) = 09C9BABB

6. `outer = SHA256(K'xor_opad || inner)` — Dış özet: bu, HMAC etiketinin kendisi.

Uyar: kısa anahtar, **Zor:** uzun anahtar, önce hashlenir ve **Uç durum:** anahtar tam blok boyunda örneklerini karşılaştırın.

OpenSSL ile HMAC-SHA-256

```
printf 'tutar=100;alici=TR00' | openssl dgst -sha256 -mac HMAC -macopt hexkey:${(openssl rand -hex 32)}
```

Şifreleme ile MAC'i birleştirmenin üç yolu

Yöntem	Nasıl?	Örnek	Değerlendirme
Önce şifrele, sonra MAC (Encrypt-then-MAC)	Önce şifrele, sonra şifreli metnin MAC'ini al	IPsec ESP, TLS'in EtM uzantısı	Doğru yol: MAC önce doğrulanır, geçersizse şifre hiç çözülmez
MAC-sonra-şifrele (MAC-then-Encrypt)	Düz metnin MAC'ini al, ikisini birlikte şifrele	Eski TLS (CBC kipleri)	Dolgu kâhinine açık; kaçının
Şifrele-ve-MAC (Encrypt-and-MAC)	Düz metni şifrele, düz metnin MAC'ini ayrıca ekle	Eski SSH	MAC düz metin hakkında bilgi sızdırabilir; kaçının

AEAD kipleri bu seçimi sizin yerinize doğru yapar. Kendi birleştirmenizi yapmak zorundaysanız (Tarif 6.18): **iki ayrı anahtar** kullanın (biri şifreleme, biri MAC için; HKDF ile tek sırdan türetilabilir), önce şifrele, sonra MAC sırasını izleyin, MAC'e IV'yi ve bağlam bilgisini de katın, karşılaştırmayı sabit zamanlı yapın.

Aynı mesajı iki sırayla da koruyup AYNI kurcalamayı uygulayın: önce-şifrele-sonra-MAC tek bir nedenle (MAC uyuşmuyor) hemen reddeder; MAC-sonra-şifrele önce çözer/dolguyu denetler, bu yüzden bozulmanın türüne göre İKİ FARKLI hata verebilir:

mesaj: "SecretPayload01" (15 bayt)

S E C R E T P a y l o a d 0 1

1. Aynı iki yapı taşı — CBC şifreleme ve bir MAC — FARKLI sırayla birleştiriliyor.

mesaj: "SecretPayload01" (15 bayt)

S E C R E T P a y l o a d 0 1

Şifrele-sonra-MAC:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 A0 6D

2. etm_protect — $ct = cbc_encrypt(pad(plaintext), ...)$,
tag = mac(mac_key, ct) (CIPHERTEXT üzerinden).

mesaj: "SecretPayload01" (15 bayt)

S E C R E T P a y l o a d 0 1

Şifrele-sonra-MAC:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 A0 6D

3. etm_unprotect — MAC tutar: unpad(cbc_decrypt(...)) ÇALIŞIR, düz metin geri verilir.

mesaj: "SecretPayload01" (15 bayt)

S E C R E T P a y l o a d 0 1

Şifrele-sonra-MAC:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 A0 6D

MAC-sonra-şifrele:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 3C E8 0B 2F 04 33 7B 57 0C

4. mte_protect — tag = mac(mac_key, plaintext) (DÜZ METİN üzerinden), sonra
 $cbc_encrypt(pad(plaintext+tag), ...)$.

mesaj: "SecretPayload01" (15 bayt)

S E C R E T P a y l o a d 0 1

Şifrele-sonra-MAC:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 A0 6D

MAC-sonra-şifrele:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 3C E8 0B 2F 04 33 7B 57 0C

5. mte_unprotect — unpad(cbc_decrypt(...)) HER ZAMAN ÇALIŞIR (MAC henüz görülmedi).

mesaj: "SecretPayload01" (15 bayt)

S E C R E T P a y l o a d 0 1

Şifrele-sonra-MAC:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 A0 6D

MAC-sonra-şifrele:

01 00 A7 72 04 07 0C 0A E8 F8 0B 19 FA 25 07 3C E8 0B 2F 04 33 7B 57 0C

6. Dolgu geçerliydi; mac KONTROL EDİLDİ ve tutuyor: düz metin geri verildi.

Zor: dolguyu bozmayan kurcalama ile Uç durum: dolguyu bozan kurcalama örneklerini yan yana çalıştırıp hangi hattın hangi noktada durduğunu karşılaştırın.

Yeniden oynatma: geçerli ama eski mesaj

Bir MAC mesajın **değiştirilmediğini** kanıtlar, ama **yeni** olduğunu kanıtlamaz. Saldırgan geçerli bir "100 TL gönder" iletisini kaydedip on kez yeniden gönderebilir; her seferinde MAC geçerlidir. Kitabın Tarif 6.21'de anlattığı savunmalar:

Yöntem	Nasıl?	Not
Sayaç	Her ileti bir önceki sayaçtan büyük bir sayı taşır; alıcı son sayacı saklar	En basit ve güvenilir; sayaç MAC'in içinde olmalı
Zaman damgası	İleti kısa bir geçerlilik penceresi içinde olmalı	Saat eşzamanlılığı gerekir; pencere içinde tekrar için ek önlem
Tek kullanımlık değer (nonce)	Sunucu her istek için yeni bir değer verir, istemci bunu MAC'e katar	Sunucu kullanılan değerleri hatırlar

Ödemede sayaç

Temassız kart işlemlerinde her işlem, kartın **işlem sayacını** (ATC) içeren bir kriptogramla imzalanır; sunucu aynı sayacı ikinci kez kabul etmez. Mobil ödeme kütüphanelerinde her tek kullanımlık anahtar da yalnız bir işlemde kullanılır ve sayaç veritabanında tutulur. [2. haftadaki](#) tehdit tablosunda "kullanım sayacının geri sarılması" tehdidinin varlık koruma sınıfının **I+** olması bu yüzdendir.

İşlenmiş örnek: HMAC'i elle hesaplamak ve çiğ etkisini görmek

Kuralları soyut bırakmadan, gerçek bir HMAC-SHA-256 hesabını uçtan uca izleyelim. Sabit (yalnız bu örnek için) 32 baytlık bir anahtar kullanıp bir ödeme talimatının HMAC'ini hesaplayalım:

HMAC-SHA-256 (gerçek çıktı)

```
ANAHTAR=0000000000000000000000000000000000000000000000000000000000000001 # 32 bayt, yalnız gösterim için sabit
printf 'tutar=100;alici=TR00' | openssl dgst -sha256 -mac HMAC -macopt hexkey:$ANAHTAR
```

```
SHA2-256(stdin)= a08fb1159871e3b8110dedf3e3524cab7bea06f4795f2119394e11c42ac5f9a1
```

Şimdi saldırgan tutarı 100'den 900'e değiştirsin; gerçek anahtarla asıl mesajın HMAC'i şu şekilde değişir:

```
printf 'tutar=900;alici=TR00' | openssl dgst -sha256 -mac HMAC -macopt hexkey:$ANAHTAR
```

```
SHA2-256(stdin)= 8a786eacbef7863d44533b9ebcbcf288e5136204080258e838e283f1f91a46be
```

Tek bir karakterlik değişiklik (1 → 9), etiketin **tamamen farklı** 32 baytlık üretti — bu, iyi bir özet fonksiyonunun "çiğ etkisi"dir (avalanche effect): girdideki tek bitlik değişiklik, çıktının yaklaşık yarısını değiştirir. Saldırgan anahtarı bilmediği için mesajı değiştirip **geçerli** bir yeni etiket üretemez; alıcı etiketleri karşılaştırıp uyumsuzluğu yakalar.

Sabit zamanlı karşılaştırma: neden `memcmp` yetmez

Alıcı tarafta HMAC doğrulaması, hesaplanan etiketle gelen etiketi **karşılaştırmayı** gerektirir. Sıradan bir `memcmp(a, b, 32)` baytları soldan sağa karşılaştırır ve **ilk farklı baytta durur**; bu, fonksiyonun çalışma süresinin kaç baytın eşleştiğine bağlı olarak değiştiği anlamına gelir. Bir ağ üzerinden yeterince çok deneme yapabilen bir saldırgan, yanıt sürelerindeki mikrosaniyelik farklardan doğru etiketi **bayt bayt** tahmin edebilir (zamanlama saldırısı, timing attack).

Yanlış ve doğru karşılaştırma

```
/* YANLIŞ: erken cikis sure farki yaratir */
if (memcmp(hesaplanan, gelen, 32) == 0) { /* kabul */ }

/* DOGRU: sabit zamanli, her zaman 32 baytin tamamini gezer */
if (CRYPTO_memcmp(hesaplanan, gelen, 32) == 0) { /* kabul */ } /* OpenSSL */
/* ya da: sodium_memcmp (libsodium), hmac.compare_digest (Python) */
```



Sık yapılan hata: `==` ya da `memcmp` ile MAC/etiket karşılaştırma

Bir web servisinin API imza doğrulama kodunda `strcmp(hesaplanan_imza, gelen_imza) == 0` gibi bir karşılaştırma görülürse bu, teorik değil **kullanılabilir** bir zamanlama saldırısı riskidir: yeterli istek sayısı (binlerce -on binlerce) saldırgan doğru imzayı bayt bayt keşfedebilir. Kural: **hiçbir** gizli değer karşılaştırması erken çıkışlı bir fonksiyonla yapılmaz.



Kural

MAC, imza, parola özeti gibi gizli değerleri her zaman sabit zamanlı bir fonksiyonla (`CRYPTO_memcmp`, `sodium_memcmp`, dilin kendi "constant-time compare" fonksiyonu) karşılaştırın. Kendi karşılaştırma fonksiyonunuzu yazmayın; bu haftanın "kendi kriptonuzu yazmayın" kuralının bir başka görünümüdür (Bölüm 1).

Üçüncü haftada AEAD'i "gizlilik + bütünlüğü tek pakette veren araç" olarak tanıttık; bu bölüm bütünlüğü **ayrı** sağlamak zorunda kaldığınızda (AEAD kullanmadığınız eski bir protokol, donanım kısıtı, vb.) doğru sırayı ve doğru karşılaştırmayı gösterdi. Sıradaki bölüm, MAC'in veremediği şeyi — **inkâr edilemezliği** — veren asimetrik araçlara geçiyor.

4. Asimetrik kriptografi: RSA ve eliptik eğriler (Tarif 7.1–7.3)

Simetrik kriptografinin tek zorluğu, iki tarafın aynı anahtarı **önceden** paylaşmasıdır. Asimetrik (açık anahtarlı) kriptografide her tarafın bir **anahtar çifti** vardır: herkese verilen **açık anahtar** ve yalnız sahibinin bildiği **özel anahtar**. Açık anahtarla şifrelenen yalnız özel anahtarla çözülür; özel anahtarla imzalanan herkesçe açık anahtarla doğrulanır.

RSA dolgusu: hangisi nerede?

dolgu, algoritmanın güvenliğini ayrılmaz parçasıdır

OAEP

RSA ile ŞİFRELEME
rastgeleleştirilmiş dolgu
ham RSA asla kullanılmaz

PSS

RSA ile İMZA
rastgeleleştirilmiş dolgu
PKCS#1 v1.5 yerine tercih

Eliptik eğride karşılıkları: X25519 (anahtar değişimi), Ed25519 (imza).

Asimetrik işlemler simetrik olanlardan yüzlerce, binlerce kat yavaştır. Bu yüzden asimetrik kriptografi yalnız iki iş için kullanılır (Tarif 7.1): **kısa bir simetrik anahtarı taşımak ya da anlaşılmak** ve **imzalamak**. Verinin kendisi her zaman simetrik şifreyle korunur (3. haftadaki hibrit şema).

RSA: doğru dolguyla

RSA'yı Rivest, Shamir ve Adleman 1977'de yayımladı. Matematiği yalın haliyle ("ders kitabı RSA") güvensizdir: aynı mesaj her zaman aynı şifreli metni verir ve matematiksel yapısı saldırılara açıktır. Güvenli kullanım, **doğru dolgu şemasına** bağlıdır — **PKCS#1** (v1.5) dolgu şeması 1991'de RSA Laboratories tarafından yayımlandı; Daniel Bleichenbacher 1998'de v1.5 şifrelemesine karşı pratik bir dolgu kâhini saldırısı yayımladı:

İş	Doğru şema	Eskimiş şema	Neden?
Şifreleme	RSA-OAEP (SHA-256)	PKCS#1 v1.5 şifreleme	Bleichenbacher'ın 1998'deki dolgu kâhini saldırısı ve sonraki yıllarda ortaya çıkan varyantları
İmza	RSA-PSS	PKCS#1 v1.5 imza	v1.5 imzası hâlâ yaygın ve kırık değil, ama PSS güvenlik kanıtı olan şemadır

Kitabın Tarif 7.10–7.13'teki "ham" RSA işlemleri, dolgusuz RSA'nın nasıl çalıştığını anlamak içindir; üretimde kullanılmaz. Tarif 7.14'teki uyarı ise hâlâ önemlidir: **imzala-ve-şifrele** birleştirmesi dikkat ister; imza, şifrelenen içeriğin alıcısına ve bağlamına bağlanmazsa, alıcı imzalı mesajı başka birine yeniden şifreleyip iletebilir.

OpenSSL 3 ile RSA-3072: anahtar, OAEP şifreleme, PSS imza

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out rsa_ozel.pem
openssl pkey -in rsa_ozel.pem -pubout -out rsa_acik.pem

# OAEP (SHA-256) ile kısa bir oturum anahtarını şifrele
openssl rand -out oturum.key 32
openssl pkeyutl -encrypt -pubin -inkey rsa_acik.pem -in oturum.key -out oturum.enc \
    -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256

# PSS ile imzala ve doğrula
openssl dgst -sha256 -sign rsa_ozel.pem -sigopt rsa_padding_mode:pss -out belge.sig belge.txt
openssl dgst -sha256 -verify rsa_acik.pem -sigopt rsa_padding_mode:pss -signature belge.sig belge.txt
```

3072 bitlik gerçek bir anahtarın matematiği elle izlenemez; aynı işlemleri (üretim, şifreleme, çözme, imzalama, doğrulama) **oyuncak asallarla** (Wikipedia'nın klasik $p=61$, $q=53$ örneği) uçtan uca izleyin:

$p = 61$ $q = 53$

1. Anahtar üretimi: iki küçük asal, $p=61$, $q=53$.

$p = 61$ $q = 53$

2. $\text{is_prime}(p)$ ve $\text{is_prime}(q)$ — ikisi de asal mı? EVET (koşul YANLIŞ, hata fırlatılmaz).

$p = 61$ $q = 53$

3. $p == q$? HAYIR — iki farklı asal.

$p = 61$ $q = 53$

$n = p \cdot q = 3233$
 $\phi = (p-1)(q-1) = 3120$

4. $n = p \cdot q = 3233$, $\phi = (p-1)(q-1) = 3120$.

$p = 61$ $q = 53$
 $n = p \cdot q = 3233$
 $\phi = (p-1)(q-1) = 3120$
 $e = 17$ $d = 2753$

5. $d = \text{modinv}(e, \phi) = 2753$ — açık anahtar ($e=17$, $n=3233$), gizli anahtar ($d=2753$, $n=3233$).

$p = 61$ $q = 53$
 $n = p \cdot q = 3233$
 $\phi = (p-1)(q-1) = 3120$
 $e = 17$ $d = 2753$

 $m = 65$ $c = 2790$

6. $\text{encrypt}(m, \text{pub})$: $0 \leq m < n$? EVET (aralık dışı değil). $c = m^e \bmod n = 65^{17} \bmod 3233 = 2790$.

$p = 61$ $q = 53$
 $n = p \cdot q = 3233$
 $\phi = (p-1)(q-1) = 3120$
 $e = 17$ $d = 2753$

 $m = 65$ $c = 2790$ $\text{decrypt}(c) = 65$

7. $\text{decrypt}(c, \text{priv})$: $c^d \bmod n = 2790^{2753} \bmod 3233 = 65$ ($= m$: true).

$p = 61$ $q = 53$
 $n = p \cdot q = 3233$
 $\phi = (p-1)(q-1) = 3120$
 $e = 17$ $d = 2753$

 $m = 65$ $c = 2790$ $\text{decrypt}(c) = 65$

 $\text{sign}(m) = 588$

8. $\text{sign}(m, \text{priv})$: $m^d \bmod n = 65^{2753} \bmod 3233 = 588$ (decrypt ile AYNI işlem, üsler sadece d).

```

p = 61          q = 53
n = p*q = 3233
phi = (p-1)(q-1) = 3120
e = 17          d = 2753

m = 65          c = 2790          decrypt(c) = 65

sign(m) = 588
verify = true

```

9. verify(m, sig, pub): $\text{sig}^e \bmod n = 65; == m (65)?$
EVET → GEÇERLİ.

Uyar: klasik örnek ile başlayıp Uç durum: imza kurcalandı ve Uç durum: yanlış açık anahtarla doğrulama örneklerinde doğrulamanın neden başarısız olduğunu izleyin.

Eliptik eğriler: aynı güvenlik, daha kısa anahtar

Eliptik eğri kriptografisi (ECC), birbirinden bağımsız olarak Neal Koblitz (1987) ve Victor Miller (1985) tarafından önerildi. 256 bitlik anahtarla 3072 bitlik RSA'nın güvenlik düzeyini sağlar; işlemleri daha hızlı, anahtarları ve imzaları çok daha kısadır.

Algoritma	İş	Not
Ed25519	İmza	Deterministik (rastgele sayı gerektirmez), hızlı, yan kanallara karşı dikkatle tasarlanmış
X25519	Anahtar anlaşması	TLS 1.3, SSH, Signal gibi protokollerin varsayılanı
ECDSA (P-256)	İmza	Yaygın ve standart; ama her imzada benzersiz ve gizli bir rastgele değer ister
ECDH (P-256)	Anahtar anlaşması	FIPS onaylı sistemlerde yaygın



ECDSA ve rastgele sayı

ECDSA her imzada gizli bir rastgele değer (k) kullanır. Aynı k iki farklı imzada kullanılırsa ya da tahmin edilebilirse, iki imzadan **özel anahtar** hesaplanabilir. 2010'da bir oyun konsolunun imza anahtarı, üreticinin her imzada aynı k 'yi kullanması yüzünden bu yolla ortaya çıktı; 2013'te bazı mobil kripto para cüzdanları, zayıf bir rastgele üreteç yüzünden aynı şekilde kayıp yaşadı. **3. haftadaki "rastgele sayılar kriptografinin görünmez temelidir"** cümlesinin en çarpıcı kanıtıdır. Ed25519 ve RFC 6979'daki deterministik ECDSA bu riski ortadan kaldırır.

OpenSSL 3 ile Ed25519 imza

```

openssl genpkey -algorithm ED25519 -out ed_ozel.pem
openssl pkey -in ed_ozel.pem -pubout -out ed_acik.pem
openssl pkeyutl -sign -inkey ed_ozel.pem -rawin -in belge.txt -out belge.ed.sig
openssl pkeyutl -verify -pubin -inkey ed_acik.pem -rawin -in belge.txt -sigfile belge.ed.sig

```

X25519/Ed25519'un içindeki işlem, eğri üzerinde **nokta toplamadır** — modüler üs alma yerine. Aynı Diffie–Hellman fikrini (Bölüm 6), aynı oyuncak ölçekte, küçük bir eğri üzerindeki nokta toplamayla görün:

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$

1. Sabit eğri: $y^2 = x^3 + 2x + 2 \pmod{17}$; taban nokta $G=(0,6)$ (eğri üzerinde bulundu).

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$

Alice

Bob

A = a*G

(0,6)

(9,1)

2. adım 1/2: en sağdaki bit 1: result = point_add(result, addend), sonra addend = point_add(addend, addend) (ikiye katlama), $k \gg= 1$.

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$

Alice

Bob

A = a*G

(6,3)

(7,6)

3. adım 2/2: en sağdaki bit 1: result = point_add(result, addend), sonra addend = point_add(addend, addend) (ikiye katlama), $k \gg= 1$.

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$

Alice

Bob

A = a*G

(6,3)

(7,6)

4. döngü bitti (kalan $k = 0$): return result = (6,3).

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$

Alice

Bob

A=(6,3)

A

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$

Alice

Bob

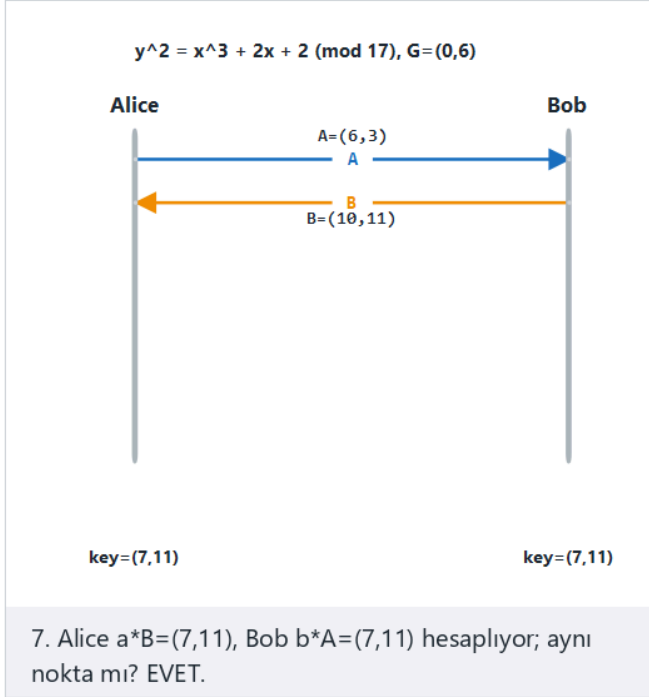
A=(6,3)

A

B=(10,11)

5. $\text{scalar_mult}(a, G)$ — Alice $A=(6,3)$ noktasını gönderiyor.

6. $\text{scalar_mult}(b, G)$ — Bob $B=(10,11)$ noktasını gönderiyor.



Zor: büyük özel değerler örneğinde adım sayısının nasıl arttığını, **Uç durum: Mallory araya giriyor** örneğinde ise kimliksiz bir eğri değişiminin de Bölüm 6'daki zayıflığı aynen taşıdığını görün.

İşlenmiş örnek: RSA-3072 ile Ed25519'u ölçmek

"ECC aynı güvenliği daha küçük anahtarlar verir" cümlesini gerçek baytlarla doğrulayalım. Her ikisi de ~ 128 bit güvenlik düzeyindedir (bkz. Bölüm 1 tablosu):

Anahtar ve imza boyutlarını ölçmek (gerçek çıktı)

```
wc -c rsa_acik.pem ed_acik.pem
openssl dgst -sha256 -sign rsa_ozel.pem -sigopt rsa_padding_mode:pss -out belge.rsa.sig belge.txt
openssl pkeyutl -sign -inkey ed_ozel.pem -rawin -in belge.txt -out belge.ed.sig
wc -c belge.rsa.sig belge.ed.sig
```

```
636 rsa_acik.pem
116 ed_acik.pem
384 belge.rsa.sig
64 belge.ed.sig
```

Açık anahtar dosyası (PEM, Base64 kodlu) RSA'da **636 bayt**, Ed25519'da **116 bayt** — yaklaşık 5,5 kat küçük. İmza RSA'da **384 bayt** (3072 bit = 384 bayt, RSA imzası tam olarak modül boyutu kadardır), Ed25519'da **64 bayt** — 6 kat küçük. Bir IoT cihazının her el sıkışmada gönderdiği veri, gömülü bir sistemin flash'ta sakladığı sertifika sayısı ya da bir blok zincirinde

her işlemin taşıdığı imza sayısı düşünüldüğünde, bu fark birikerek büyür; ECC'nin mobil/gömülü sistemlerde ve yeni protokollerde (TLS 1.3, SSH, Signal) varsayılan olmasının nedeni budur.

İşlenmiş örnek: OAEP'in rastgeleliği neden gerekli?

Bölüm 2'de ECB'nin aynı düz metin bloğu için aynı şifreli metni ürettiğini ve bunun bir desen sızıntısı olduğunu gördük. "Ders kitabı RSA" (ham, dolgusuz) da aynı hastalığı taşır: aynı mesaj her zaman aynı şifreli metni verir. OAEP bunu, şifrelemeden önce mesaja **rastgele** bir değer karıştırarak çözer. Aynı mesajı iki kez OAEP ile şifreleyelim:

OAEP: aynı mesaj, iki farklı şifreli metin (gerçek çıktı)

```
echo MERHABA > kisa.txt
openssl pkeyutl -encrypt -pubin -inkey rsa_acik.pem -in kisa.txt -out c1.bin \
  -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
openssl pkeyutl -encrypt -pubin -inkey rsa_acik.pem -in kisa.txt -out c2.bin \
  -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
cmp c1.bin c2.bin && echo "AYNI" || echo "FARKLI"
```

```
c1.bin c2.bin differ: char 1, line 1
FARKLI
```

Aynı açık anahtar, aynı düz metin, ama **farklı** şifreli metin — çünkü OAEP her şifrelemede yeni bir rastgele değer kullanır (Bölüm 1'deki "rastgele sayılar kriptografinin görünmez temelidir" ilkesinin RSA'daki karşılığı). Dolgusuz ("ham") RSA ile aynı deneyi yaparsanız iki çıktı **birebir aynı** olur; matematiksel olarak $c = m^e \bmod n$ deterministik bir işlemdir. Bu yüzden ham RSA hiçbir zaman doğrudan şifreleme için kullanılmaz.



Sık yapılan hata: RSA anahtarını hem şifreleme hem imza için kullanmak

Aynı RSA anahtar çiftini hem "veriyi şifrele" hem "veriyi imzala" için kullanmak, iki farklı matematiksel işlemi aynı anahtara bindirir; bazı saldırı senaryolarında (ör. bir imzalama isteğinin, dikkatlice seçilmiş bir "düz metni" imzalıyor gibi görünüp aslında bir şifre çözme isteğine dönüştürülmesi) bu paylaşım kötüye kullanılabilir. Kural: **anahtar kullanım amacı tektir** — sertifikanın `keyUsage` alanı da (Bölüm 8) bunu zorunlu kılar: `keyEncipherment` ile `digitalSignature` ayrı bitlerdir, bir anahtar ikisini birden taşımamalıdır.



Kural

Şifreleme için her zaman **OAEP**, imza için her zaman **PSS** kullanın; **PKCS#1** v1.5 yalnız eski sistemlerle uyumluluk zorunluysa ve o zaman da yalnız imza için (asla yeni şifreleme tasarımı) düşünülür. Yeni tasarımda mümkünse RSA yerine Ed25519/X25519'u tercih edin.

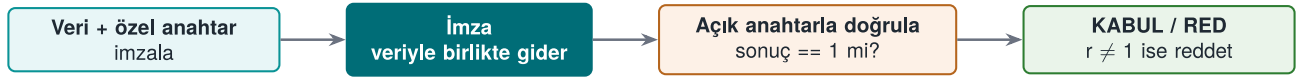
Üçüncü haftada TLS'in anahtar kurulumunda asimetrik kriptografiyi (o zaman RSA ya da ECDHE) bir kara kutu gibi kullanmıştık. Bu bölüm o kara kutunun içini açtı: hangi dolgu, hangi eğri, hangi boyut. Sıradaki bölüm bu araçların **imza** ayağını derinleştiriyor; sonra Bölüm 6'da **anahtar değişimi** ayağına geçiyoruz.

5. Dijital imza: oluşturma, doğrulama ve kullanım yerleri

Dijital imza, bir mesajın (1) belirli bir özel anahtarın sahibi tarafından oluşturulduğunu ve (2) sonradan değiştirilmediğini kanıtlar. MAC'ten farkı, doğrulamanın **açık anahtarla** yapılmasıdır: herkes doğrulayabilir ama yalnız özel anahtarın sahibi imzalayabilir. Bu yüzden imza **inkâr edilemezlik** sağlar (STRIDE'daki R), MAC sağlamaz.

Dijital imza: oluşturma ve doğrulama

en sık hata: if(r) ile denetlemek



API 1=başarı, 0=başarısız, <0=hata döner; if(r) hatayı da “doğru” sayar.

İmza nerelerde kullanılır?

Kullanım	Ne imzalanır?	Bu dersteki yeri
Yazılım güncellemesi	Güncelleme paketi	1. hafta "Kasa" örneği, T6 tehdidi
Kod imzalama	Çalıştırılabilir dosya, mobil uygulama paketi	6. hafta imza doğrulaması
Sertifika	Açık anahtar + kimlik bilgisi (CA imzalar)	Bu hafta PKI
TLS el sıkışması	El sıkışma özeti (sunucu imzalar)	3. hafta CertificateVerify
Belge ve işlem	Sözleşme, ödeme onayı	Elektronik imza mevzuatı

Kodda imza doğrulama: OpenSSL EVP

Ed25519 imzasını doğrulamak (OpenSSL 3)

```

#include <openssl/evp.h>
#include <openssl/pem.h>

/* 1: geçerli, 0: geçersiz, -1: hata. Güncelleme paketini ÇALIŞTIRMADAN önce çağrılır. */
int imza_dogrula(EVP_PKEY *acik, const unsigned char *veri, size_t n,
                const unsigned char *imza, size_t imza_n)
{
    EVP_MD_CTX *ctx = EVP_MD_CTX_new();
    int sonuc = -1;
    if (ctx == NULL) return -1;
    if (EVP_DigestVerifyInit(ctx, NULL, NULL, NULL, acik) == 1) /* Ed25519: özet NULL */
        sonuc = EVP_DigestVerify(ctx, imza, imza_n, veri, n); /* 1 ya da 0 */
    EVP_MD_CTX_free(ctx);
    return sonuc == 1 ? 1 : (sonuc == 0 ? 0 : -1);
}
  
```



İmza doğrulamasının üç tuzağı

1. **Dönüş değeri:** `EVP_DigestVerify` başarıda 1, imza geçersizse 0, hata durumunda negatif döner. `if (sonuc)` gibi bir denetim, **negatif hata değerini "geçerli"** sayar. Denetim her zaman `== 1` olmalıdır.
2. **Neyin imzalandığı:** İmza yalnız paketin içeriğini kapsıyorsa, saldırgan **eski ve zafiyetli** ama geçerli imzalı bir paketi geri yükleyebilir (sürüm geri alma). Sürüm numarası ve hedef ürün de imzalı içeriğin parçası olmalı ve istemci eski sürümü reddetmeli.
3. **Hangi anahtarla:** İmzayı, paketin **kendi içinde gelen** bir açık anahtarla doğrulamak hiçbir şey kanıtlamaz. Doğrulama anahtarı uygulamaya önceden gömülü (ve bütünlüğü korunan) ya da güvenilir bir zincirden gelmelidir.

İşlenmiş örnek: imza doğrulamasını üç durumda komut satırından izlemek

Yukarıdaki C kodunun mantığını, derlemeye gerek kalmadan `openssl pkeyutl -verify` ile de görebilirsiniz — CLI da aynı üç sonucu (geçerli / geçersiz / hata) üretir:

1) Doğru belge, doğru imza

```
openssl pkeyutl -verify -pubin -inkey ed_acik.pem -rawin -in belge.txt -sigfile belge.ed.sig
echo "cikis kodu: $?"
```

```
Signature Verified Successfully
cikis kodu: 0
```

2) Belge değişti, imza aynı

```
echo "MERHABA DEGISTI" > belge2.txt
openssl pkeyutl -verify -pubin -inkey ed_acik.pem -rawin -in belge2.txt -sigfile belge.ed.sig
echo "cikis kodu: $?"
```

```
Signature Verification Failure
cikis kodu: 1
```

Belgenin tek bir baytı bile değişince doğrulama **başarısız** oldu ve çıkış kodu 0'dan 1'e döndü — güncelleme mekanizmanız bu kodu denetlemiyorsa (ya da C kodunda `EVP_DigestVerify`'in dönüşünü `if (sonuc)` ile yorumluyorsa), kurcalanmış bir paketi **çalıştırılabilir** hâle getirmiş olursunuz.



code/week-10/02-signature-verification/demo.sh neden ECDSA (P-256) kullanıyor, Ed25519 değil

Yukarıdaki komutlar Ed25519 ile (`openssl pkeyutl -sign -rawin`); bu, OpenSSL 3.x ister. Ders deposundaki `demo.sh` yerine `openssl dgst -sha256 -sign` ile **ECDSA (P-256)** kullanır, çünkü bu komut OpenSSL 1.1.1'de de 3.x'te de aynı şekilde çalışır (WSL/Ubuntu 20.04 gibi eski OpenSSL'li ortamlarda bile). Mantık — imzala/doğrula/kurcala/reddet — birebir aynıdır; yalnız algoritma farklıdır.

Aynı mantığı, gerçek bir imza üzerinde altı senaryoda (doğru imza, kurcalanmış belge, yanlış anahtarla imzalanmış bir mesaj) izleyin:

belge = U p d a t e P a c k a g e 0 1

1. Belge, 15 bayt: "UpdatePackage01".

belge = U p d a t e P a c k a g e 0 1

imza=1287

2. openssl dgst -sign signer.key ... — yayıncı imzalıyor, sig=1287.

belge = U p d a t e P a c k a g e 0 1

imza=1287

doğrula(signer.pub) = true

3. openssl dgst -verify signer.pub -signature ... — sonuç: GEÇERLİ.

Uyar: doğru imza, doğru anahtar, Zor: belge imzadan sonra kurcalandı ve Uç durum: saldırganın kendi anahtarıyla imzalanmış örneklerini karşılaştırın.

Sürüm geri alma: imza geçerli ama içerik eski

"İmzanın kapsamı" tuzağını somutlaştıralım. Diyelim ki bir uygulama güncelleme sunucusu her sürümü ayrı ayrı imzalar:

Sürüm	İçerik	İmza
v1.0	(bilinen bir güvenlik açığı içerir)	imza_v1 — hâlâ matematiksel olarak geçerli
v2.0	açığı kapatır	imza_v2 — geçerli

Sunucu v2.0'ı yayımladıktan sonra v1.0'in imzası **iptal edilmez** — imzalar CRL/OCSP gibi bir iptal mekanizmasına sahip değildir (bu, Bölüm 10'da sertifikalar için göreceğimiz mekanizmadır). Saldırgan, ele geçirdiği v1.0 paketini ve imza_v1'i istemciye sunarsa, istemci yalnız "imza geçerli mi?" diye sorup **evet** cevabını alır ve zafiyetli sürümü kurar. Bu yüzden imzalanan veri yalnız dosya içeriği değil, **sürüm numarası + hedef ürün adı** olmalı ve istemci ayrıca "gördüğüm en yüksek sürümden daha eski bir sürümü kabul etme" kuralını **kendi başına** uygulamalıdır — imza bunu sizin yerinize yapmaz.



Sık yapılan hata: imzayı yalnız 'var mı yok mu' diye sormak

Bir istemcinin yalnız imzalandı_mi(paket) gibi ikili bir soru sorması (hangi anahtarla, hangi sürüm için, hangi hedef için imzalandığını sormadan) sürüm geri alma ve **hedef karıştırma** (bir ürün için imzalanmış paketin başka bir ürüne sunulması) saldırılarına açık kapı bırakır.



Kural

İmzalanan veri her zaman **bağlamını** taşıyın: sürüm, hedef ürün/cihaz, zaman damgası ya da geçerlilik penceresi. İstemci yalnız "imza geçerli mi" değil, "bu imza **bu** bağlam için ve **en güncel** sürüm için mi" diye sormalıdır.

Birinci haftadaki "Kasa" örneğinde bir güncelleme paketinin imzasız kabul edilmesinin T6 tehdidine yol açtığını görmüştük; bu bölüm imzanın **var olmasının bile** tek başına yetmediğini, kapsamının da doğru tasarlanması gerektiğini gösterdi. Sıradaki bölüm, imza ve asimetrik kriptografiyi **anahtar değişiminde** nasıl kullandığımızı — ve kullanmazsak neyin ters gideceğini — işliyor.

6. Anahtar değişimi: Diffie–Hellman ve araya girme (Tarif 8.16–8.18)

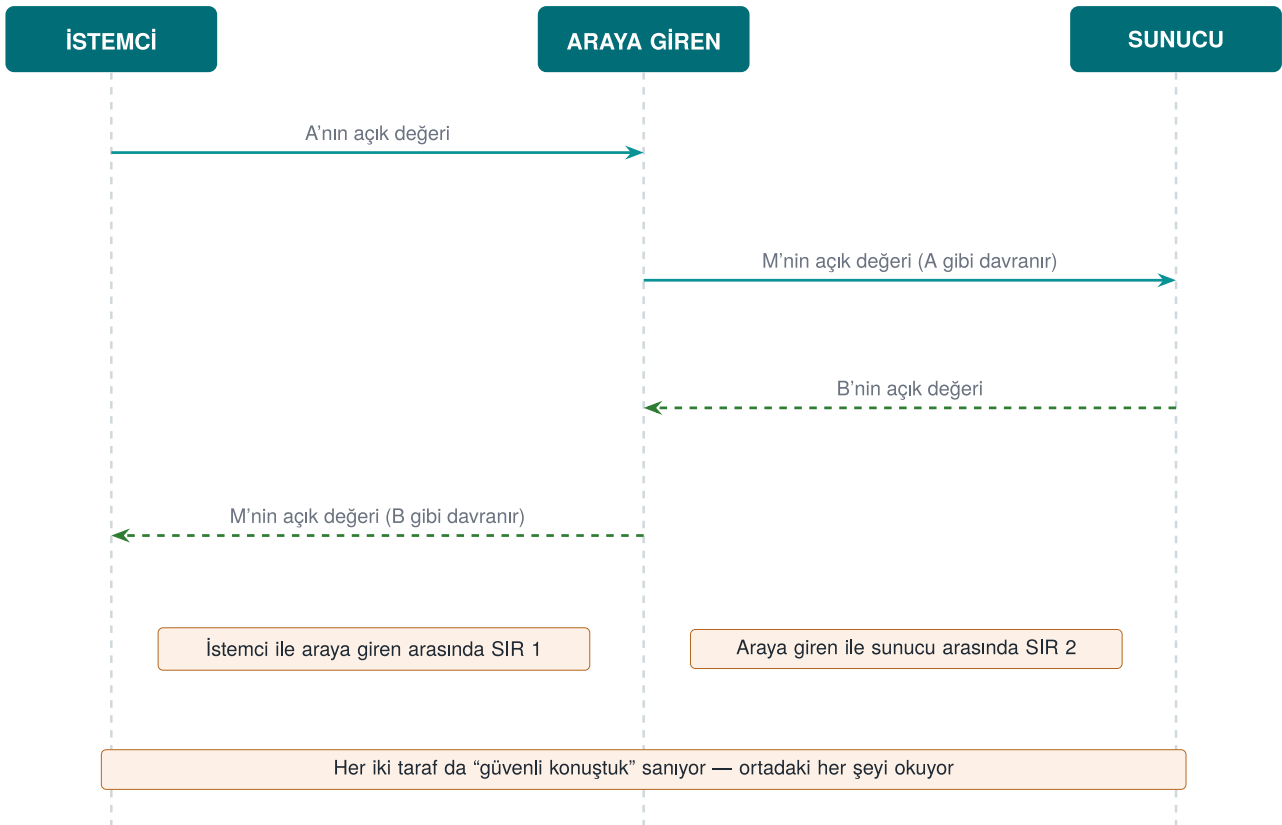
Diffie–Hellman (DH), iki tarafın güvensiz bir kanal üzerinden, hiç önceden paylaşılmış sır olmadan ortak bir sır üzerinde anlaşmasını sağlar. Her taraf bir özel değer seçer, bundan türettiği açık değeri gönderir; iki taraf da karşısındakinin açık değeri ve kendi özel değeriyle **aynı** ortak sırrı hesaplar. Dinleyen biri açık değerleri görür ama ortak sırrı hesaplayamaz. Bugün kullanılan biçimi eliptik eğri üzerindeki **ECDH**'dir (X25519).

Kimliksiz DH'nin zayıflığı: araya girme

DH **kimin** ile anlaşıldığını söylemez. Kanalin ortasındaki bir saldırgan, iki tarafla **ayrı ayrı** DH yapar:

Kimliksiz Diffie–Hellman: araya girme

DH kiminle anlaştığınızı SÖYLEMEZ



Çözüm: DH'yi kimlik kanıtıyla bağla (sertifika / imzalı DH) — TLS 1.3 bunu yapar.

İki taraf da "güvenli bir kanalım var" sanır; aslında her mesaj saldırganın elinden geçer. Aşağıda aynı saldırıyı küçük sayılarla (klasik $p=23$, $g=5$ örneği) uçtan uca izleyin — Alice ve Bob'un normal değişimiyle, sonra Mallory araya girince iki

tarafın **farklı** anahtarlara ulaştığını görün:

$p=23, g=5$

1. Ortak parametreler: $p=23$ (mod), $g=5$ (üreteç).

$p=23, g=5$

Alice

Bob

$A = g^a \text{ mod } p$

1

2

2. adım 1/3: en sağdaki bit 0: çarpma yok, sonra $\text{base} = \text{base}^2 \text{ mod } p$, $\text{exp} \gg= 1$.

$p=23, g=5$

Alice

Bob

$A = g^a \text{ mod } p$

2

4

3. adım 2/3: en sağdaki bit 1: $\text{result} = \text{result} * \text{base} \text{ mod } p$, sonra $\text{base} = \text{base}^2 \text{ mod } p$, $\text{exp} \gg= 1$.

$p=23, g=5$

Alice

Bob

$A = g^a \text{ mod } p$

8

16

4. adım 3/3: en sağdaki bit 1: $\text{result} = \text{result} * \text{base} \text{ mod } p$, sonra $\text{base} = \text{base}^2 \text{ mod } p$, $\text{exp} \gg= 1$.

$p=23, g=5$

Alice

Bob

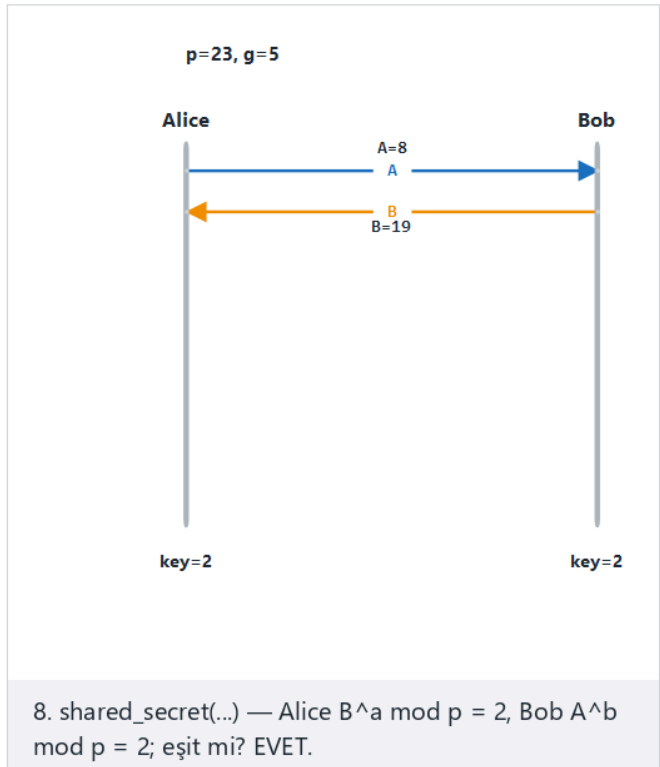
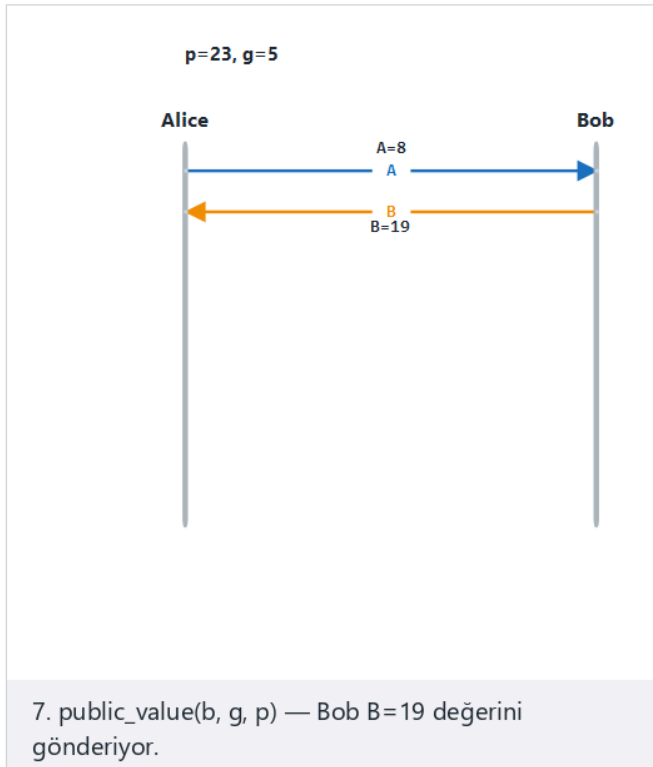
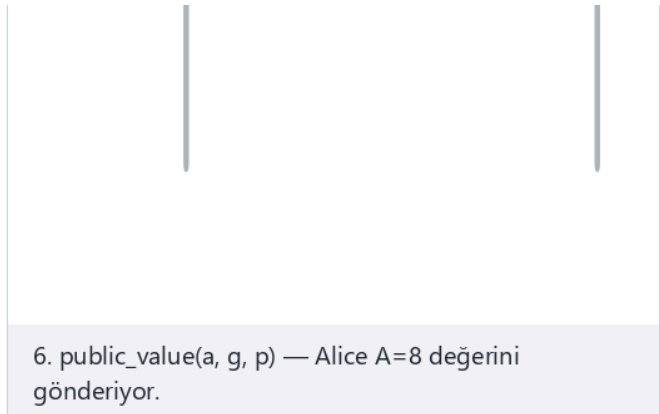
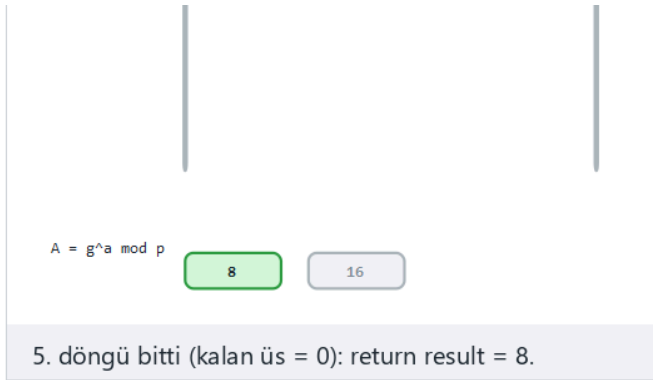
$p=23, g=5$

Alice

Bob

$A=8$

A



Uyar: klasik örnek ile dürüst değişimi, **Uç durum:** Mallory araya giriyor ile saldırıyı karşılaştırın.

Çözüm: kimliği doğrulanmış anahtar değişimi

Açık değerler, kimliği önceden bilinen bir anahtarla **imzalanır** (Tarif 8.18'deki "DH ve DSA birlikte" fikrinin güncel hali: ECDHE + Ed25519/ECDSA/RSA-PSS). TLS 1.3'te sunucu, geçici (ephemeral) ECDHE açık değerini de içeren el sıkışma özetini sertifikasındaki özel anahtarla imzalar (CertificateVerify); istemci sertifikayı zincir ve ad açısından doğrular (3. hafta). Böylece:

- **Kimlik:** İmza, açık değeri gönderenin gerçekten sertifikanın sahibi olduğunu kanıtlar.
- **İleri gizlilik:** DH değerleri her oturum için yeniden üretilir ve silinir; uzun ömürlü imza anahtarları ileride çalınsa bile geçmiş oturumların anahtarları hesaplanamaz (3. hafta).

PKI olmadan kimlik

Her senaryoda bir sertifika otoritesi yoktur. Kitabın Tarif 8.19'da anlattığı "PKI olmadan kimlik doğrulama" yolları bugün de kullanılır: ilk bağlantıda anahtarı öğrenip sonra değişirse uyarmak (TOFU, SSH), anahtarın parmak izini ayrı bir kanaldan doğrulamak (güvenli mesajlaşmadaki "güvenlik numarası" karşılaştırması) ya da her iki tarafın önceden paylaştığı bir paroladan kimlik doğrulamalı anahtar değişimi (PAKE, Tarif 8.15; bugün OPAQUE, SPAKE2 gibi protokoller).

İşlenmiş örnek: X25519 ile iki tarafın aynı sırra ulaşması

Diffie–Hellman'ın "iki taraf, önceden paylaşılmış sır olmadan aynı ortak değere ulaşır" cümlesini gerçek anahtarlarla doğrulayalım. Alice ve Bob birbirinden bağımsız birer X25519 anahtar çifti üretir, yalnız **açık** anahtarlarını değiş tokuş eder:

X25519 anahtar anlaşması: Alice ve Bob (gerçek çıktı)

```
openssl genpkey -algorithm X25519 -out alice.key
openssl genpkey -algorithm X25519 -out bob.key
openssl pkey -in alice.key -pubout -out alice.pub
openssl pkey -in bob.key -pubout -out bob.pub

# Alice: kendi özel anahtarı + Bob'un acik anahtarı
openssl pkeyutl -derive -inkey alice.key -peerkey bob.pub -out alice_sir.bin
# Bob: kendi özel anahtarı + Alice'in acik anahtarı
openssl pkeyutl -derive -inkey bob.key -peerkey alice.pub -out bob_sir.bin

xxd -p alice_sir.bin
xxd -p bob_sir.bin
cmp alice_sir.bin bob_sir.bin && echo "AYNI" || echo "FARKLI"
```

```
8c53744a000c1a6ffc2183a0c134c2cf8167110314e8a06dd3d453080bbc1376
8c53744a000c1a6ffc2183a0c134c2cf8167110314e8a06dd3d453080bbc1376
AYNI
```

İki taraf, birbirinin **hiç görmediği** özel anahtarını bilmeden, ağdan yalnız açık anahtarları geçirerek **aynı** 32 baytlık ortak sırra ulaştı. Bu ortak sır, doğrudan veri şifrelemede kullanılmaz; bir KDF'den ([HKDF, 3. haftadaki anahtar türetme konusu](#)) geçirilip oturum anahtarları buradan türetilir. Dinleyen bir üçüncü taraf yalnız `alice.pub` ve `bob.pub` 'ı görür; bunlardan ortak sırrı hesaplamak (eliptik eğri ayrık logaritma problemini çözmeden) mümkün değildir.

Sayısal örnek: araya girme kaç adımda kurulur?

Kimliksiz DH'de araya girme saldırısı **iki ayrı** DH anlaşması gerektirir — saldırganın hesap yükü kurbanlarınkinden farklı değildir, yalnız iki kat işlem yapar:

1. Saldirgan (M) Alice'e bağlanır, kendi açık değerini `pub_M1` olarak gönderir; Alice bunu Bob'un açık değeri sanır.
2. M, Bob'a bağlanır, `pub_M2` gönderir; Bob bunu Alice'in açık değeri sanır.
3. M, Alice ile `sir_A = DH(özel_M1, pub_Alice)`, Bob ile `sir_B = DH(özel_M2, pub_Bob)` hesaplar — **iki farklı** ortak sır elde eder.
4. Alice "Bob"a (aslında M'ye) `sir_A` ile şifreli mesaj gönderir; M bunu çözer, okur, `sir_B` ile yeniden şifreleyip gerçek Bob'a iletir. Her iki taraf da kendi hesapladığı sırrın **doğru karşı tarafa** paylaşıldığını sanır.

Saldirganın ek maliyeti yalnız **iki kat X25519 işlemi** (mikrosaniyeler mertebesinde) — kimlik doğrulaması olmadığında bu saldırı hesaplama açısından neredeyse bedavadır. Bu yüzden imzasız DH, TLS 1.3 gibi hiçbir modern protokolde tek

başına kullanılmaz.



Sık yapılan hata: DH çıktısını doğrudan anahtar olarak kullanmak

`pkeyutl -derive`'in ürettiği ham baytları (yukarıdaki örnekte `alice_sir.bin`) doğrudan AES anahtarı olarak kullanmak yaygın bir hatadır. Ham DH çıktısı **tekdüze rastgele** değildir (eğrinin matematiksel yapısından gelen önyargılar taşır) ve tek bir anahtar üretir; oysa genelde şifreleme ve MAC için **ayrı** anahtarlar gerekir (Bölüm 3). Kural: DH çıktısını her zaman bir KDF'den (HKDF-SHA-256) geçirip ihtiyaç kadar anahtar türetin.



Kural

Kimliği doğrulanmamış hiçbir DH/ECDH sonucuna güvenmeyin; açık değerler mutlaka imzalanmalı ya da önceden doğrulanmış bir sertifikanın parçası olmalıdır (TLS 1.3'ün `CertificateVerify`'i). Ham DH çıktısını asla doğrudan şifreleme anahtarı olarak kullanmayın; HKDF'den geçirin.

Üçüncü haftada TLS 1.3 el sıkışmasında "eğer aradaki biri anahtarları değiştirebilseydi ne olurdu" sorusunu sormuştuk; bu bölüm o soruyu somut adımlarla cevapladı. Sıradaki üç bölüm (7–9), imzalı DH'nin dayandığı kimlik doğrulama altyapısını — PKI'yı — sıfırdan kuruyor.

7. Açık anahtar altyapısı (PKI): kim kime güvenir? (Tarif 10.1)

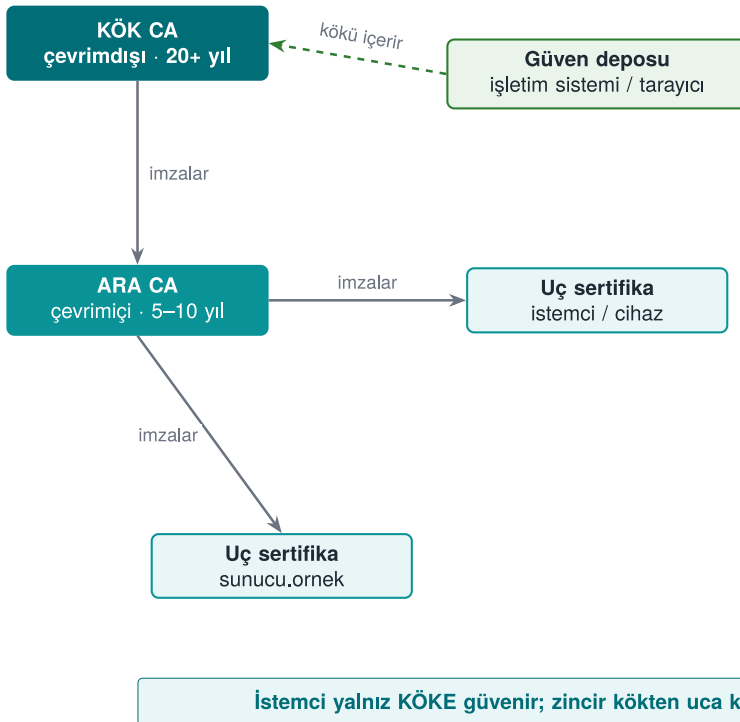
Önceki bölüm açık anahtarların imzayla kimliğe bağlanması gerektiğini gösterdi. Ama bu bir soru doğurur: sunucunun açık anahtarının **gerçekten** o sunucuya ait olduğunu nereden bileceğiz? **Açık anahtar altyapısı** (PKI), bu soruya güvenirilen üçüncü taraflar aracılığıyla cevap veren kurallar, roller ve belgeler bütünüdür.

Bileşenler

Bileşen	Görevi	Örnek
Sertifika otoritesi (CA)	Kimliği doğrulanmış bir varlığın açık anahtarını imzalayarak sertifika çıkarır	Ticari ve kamu CA'ları, kurum içi CA
Kayıt otoritesi (RA)	Sertifika isteyen kimliğini doğrular; CA'ya "bu kişiye sertifika ver" der	Kurumun insan kaynakları, bir alan adının sahipliğini denetleyen otomatik sistem (ACME)
Kök CA	Zincirin tepesindeki, kendini imzalayan CA; genellikle çevrimdışı tutulur	İşletim sistemi ve tarayıcıların güven deposundaki kökler
Ara CA	Kök tarafından imzalanmış, günlük sertifikaları çıkaran CA	Kök çevrimdışı kalabilsin diye
Uç (leaf) sertifika	Bir sunucuya, kişiye ya da cihaza ait sertifika	<code>sunucu.ornek</code> için TLS sertifikası
Güven deposu (trust store)	Güvenilen kök sertifikaların listesi	İşletim sistemi deposu, uygulamaya gömülü depo
İptal hizmetleri	İptal edilmiş sertifikaları bildirir	CRL dağıtım noktası, OCSP yanıtlayıcı

PKI: kim kime güvenir?

güven kökten uca akar



Neden ara CA? Kök CA'nın özel anahtarı ele geçirilirse **bütün** zincir güvenilmez olur ve kökü dünyadaki bütün güven depolarından çıkarmak yıllar sürer. Bu yüzden kök anahtar bir HSM'de, çevrimdışı ve fiziksel olarak korunan bir yerde

durur; yalnız ara CA sertifikalarını imzalamak için, tören (key ceremony) denen denetimli toplantılarda kullanılır. Günlük işi ara CA yapar; ele geçirilirse yalnız o ara CA iptal edilir.

Sertifika doğrulama: dört soru (Tarif 10.4)

Bir istemci bir sertifikayı kabul etmeden önce şunları sorar:

1. **Zincir:** Uç sertifikadan başlayarak her sertifikanın imzası, bir üsttekinin açık anahtarıyla doğrulanıyor mu ve zincir güven deposundaki bir köke ulaşıyor mu?
2. **Geçerlilik:** Zincirdeki her sertifikanın geçerlilik tarihleri içinde miyiz?
3. **Kullanım:** Sertifika bu iş için mi çıkarılmış? (Anahtar kullanımı ve genişletilmiş anahtar kullanımı alanları; ara CA'da `CA:TRUE` temel kısıtı.)
4. **Ad ve iptal:** Sertifika **bu** sunucu için mi (SAN)? İptal edilmiş mi?

Üçüncü haftada ilk ve dördüncü sorunun kodda nasıl sorulduğunu gördük (`SSL_CTX_set_verify`, `SSL_set1_host`). Bu bölümde aynı zinciri kendimiz kuruyoruz.

İşlenmiş örnek: "dört soru"yu komut satırında sormak

Bölüm 9'da kuracağımız kök → ara → sunucu zincirini kullanarak, bir istemcinin "dört soru"yu nasıl **somut olarak** sorduğunu görelim (bu üç dosya henüz elinizde değilse, Bölüm 9'u önce çalıştırın; burada yalnız aynı dosyaları farklı açılardan okuyoruz).

Soru 1 — Zincir: her sertifikanın `issuer` alanı, bir üsttekinin `subject` alanıyla **birebir** eşleşmeli:

```
openssl x509 -in sunucu.crt -noout -issuer -subject
openssl x509 -in ara.crt -noout -issuer -subject
openssl x509 -in kok.crt -noout -issuer -subject
```

```
issuer=CN=CEN429 Lab Ara CA      subject=CN=localhost
issuer=CN=CEN429 Lab Kok CA      subject=CN=CEN429 Lab Ara CA
issuer=CN=CEN429 Lab Kok CA      subject=CN=CEN429 Lab Kok CA
```

Zincir okunur: `sunucu` 'nın `issuer`'ı = `ara` 'nın `subject`'i; `ara` 'nın `issuer`'ı = `kok` 'un `subject`'i; `kok` 'un `issuer`'ı **kendisi** (kök, kendi kendini imzalar — bu yüzden zincir burada durur).

Soru 2 — Geçerlilik:

```
openssl x509 -in sunucu.crt -noout -dates
openssl x509 -in sunucu.crt -noout -checkend 0
```

```
notBefore=Sep 23 06:01:46 2026 GMT
notAfter=Dec 22 06:01:46 2026 GMT
Certificate will not expire
```

`-checkend 0`, "şu andan itibaren 0 saniye içinde süresi dolacak mı?" diye sorar; TLS istemcileri bu denetimi her bağlantıda otomatik yapar.

Soru 3 — Kullanım:

```
openssl x509 -in sunucu.crt -noout -ext basicConstraints,keyUsage,extendedKeyUsage
openssl x509 -in ara.crt -noout -ext basicConstraints,keyUsage
```

```
X509v3 Basic Constraints: CA:FALSE
X509v3 Key Usage: critical / Digital Signature
X509v3 Extended Key Usage: TLS Web Server Authentication
---
X509v3 Basic Constraints: critical / CA:TRUE, pathlen:0
X509v3 Key Usage: critical / Certificate Sign, CRL Sign
```

`sunucu.crt` 'nin `CA:FALSE` olması, bu sertifikanın **başka sertifika imzalamayacağını** garanti eder — bir saldırgan bu sertifikanın özel anahtarını ele geçirse bile, onunla yeni bir sertifika üretip imzalamaz. `ara.crt` ise tam tersi: `CA:TRUE` ve `pathlen:0` (altında başka bir ara CA olamaz, yalnız uç sertifika imzalayabilir).

Soru 4 — Ad: SAN alanı `-ext subjectAltName` ile yukarıda zaten görüldü (`DNS:localhost`, `IP Address:127.0.0.1`); istemci bağlandığı adı bu listeyle karşılaştırır (Bölüm 8'de ayrıntılı işleniyor).

Bir istemci kütüphanesi (OpenSSL, Windows CryptoAPI, Java'nın `X509TrustManager` 'ı) bu dört soruyu **sizin yerinize** sorar; `verify` fonksiyonlarını (3. haftadaki `SSL_CTX_set_verify`) devre dışı bırakmak ya da hataları yutmak, bu dört sorunun hiç sorulmaması anlamına gelir.



Sık yapılan hata: 'bağlandı, demek ki güvenli' varsayımı

Bir geliştiricinin "TLS bağlantısı kuruldu, o zaman sertifika doğrulanmış olmalı" varsayımı yanlıştır. Bağlantı, doğrulama **kapatılmışsa** da kurulur (3. haftadaki `SSL_VERIFY_NONE` / `InsecureSkipVerify` hatası). Bağlantının kurulmuş olması, dört sorunun **sorulduğunun** kanıtı değildir; kodda doğrulamanın açık olduğunu **ayrıca** denetlemek gerekir.



Kural

Bir istemci kütüphanesi kurarken doğrulama modunu (`SSL_VERIFY_PEER`, `CERT_REQUIRED`, vb.) **açıkça** ayarlayın ve bir testte bilerek geçersiz bir sertifika sunup bağlantının **reddedildiğini** doğrulayın — yalnız "geçerli sertifikayla bağlanabiliyorum" testi yetmez, "geçersiz sertifikayla bağlanamıyorum" testi de gerekir.

Bir önceki bölümde imzalı DH'nin "kimin ile konuştuğumu" imza ile kanıtladığını gördük; bu bölüm o imzanın **kimin** imzası olduğuna nasıl güvendiğimizi (PKI, CA hiyerarşisi) açıkladı. Sıradaki iki bölüm, bu güveni taşıyan **belgenin kendisine** (X.509 sertifikası) ve onu **elle kurmaya** bakıyor.

8. X.509 sertifikasının yapısı

Bir X.509 v3 sertifikası, **imzalanan kısım** (TBSCertificate) ile CA'nın bu kısım üzerindeki **imzasından** oluşur:

X.509 sertifikasında ne var?

doğrulama bu alanların HEPSİNİ denetler

Konu (Subject) + SAN — kime ait — ad denetimi SAN'a bakar, CN'e değil

Açık anahtar (SPKI) — pinning bunu sabitler

Geçerlilik aralığı — notBefore / notAfter

Uzantılar — basicConstraints (CA:TRUE), KeyUsage, EKU

CA imzası — zincirin bir üst halkası tarafından

Yalnız “imza geçerli” demek yetmez: ad, süre, amaç ve iptal de denetlenir.

Alan	Anlamı
Sürüm	v3 (uzantılar için gerekli)
Seri numarası	CA içinde benzersiz; iptal listeleri bu numarayı kullanır
İmza algoritması	Ör. <code>ecdsa-with-SHA256</code> , <code>sha256WithRSAEncryption</code>
Düzenleyen (Issuer)	Sertifikayı imzalayan CA'nın adı
Geçerlilik	<code>notBefore</code> – <code>notAfter</code>
Konu (Subject)	Sertifikanın sahibi
Konu açık anahtar bilgisi (SPKI)	Algoritma + açık anahtar; 3. haftadaki sabitleme bunun özetini kullanır
Uzantılar	Temel kısıtlar (<code>CA:TRUE/FALSE</code> , yol uzunluğu), anahtar kullanımı, genişletilmiş anahtar kullanımı (sunucu/istemci kimlik doğrulama, kod imzalama), SAN (alternatif adlar: DNS adları, IP), CRL dağıtım noktası, OCSP adresi (AIA)

Sertifikalar binary **DER** kodlamasıyla ya da onun Base64 ile sarılmış metin biçimi olan **PEM** ile saklanır (Tarif 7.16, 7.17): -
 -----BEGIN CERTIFICATE----- ile başlayan dosyalar PEM'dir.



Ad denetimi CN'ye değil SAN'a yapılır

Eski uygulamalar sunucu adını konunun **CN** (Common Name) alanında arardı. Bugün tarayıcılar ve RFC 6125 yalnız **SAN** uzantısına bakar; SAN'ı olmayan bir sunucu sertifikası modern istemcilerde reddedilir. Kendi sertifikalarınızı üretirken SAN eklemeyi unutmayın.

İşlenmiş örnek: gerçek bir sertifikayı alan alan okumak

Bölüm 9'da üreteceğimiz `sunucu.crt`'nin `openssl x509 -text` çıktısını, yukarıdaki tablodaki her alanla eşleştirerek okuyalım (bu tam olarak `openssl x509 -in sunucu.crt -noout -text` komutunun gerçek çıktısıdır):

openssl x509 -in sunucu.crt -noout -text (gerçek çıktı)

```
Certificate:
Data:
  Version: 3 (0x2)
  Serial Number:
    35:e2:f6:8d:d2:e3:90:f3:74:ff:16:a9:ec:5f:07:e9:21:69:76:e7
  Signature Algorithm: ecdsa-with-SHA256
  Issuer: CN=CEN429 Lab Ara CA
  Validity
    Not Before: Sep 23 06:01:46 2026 GMT
    Not After : Dec 22 06:01:46 2026 GMT
  Subject: CN=localhost
  Subject Public Key Info:
    Public Key Algorithm: id-ecPublicKey
      Public-Key: (256 bit)
      ASN1 OID: prime256v1
      NIST CURVE: P-256
  X509v3 extensions:
    X509v3 Basic Constraints:
      CA:FALSE
    X509v3 Key Usage: critical
      Digital Signature
    X509v3 Extended Key Usage:
      TLS Web Server Authentication
    X509v3 Subject Alternative Name:
      DNS:localhost, IP Address:127.0.0.1
    X509v3 Subject Key Identifier:
      EA:59:FF:05:1B:BA:D2:ED:D3:B8:2D:CF:12:32:F6:52:27:CD:98:87
    X509v3 Authority Key Identifier:
      F5:BA:39:86:1F:A1:D0:79:C8:DA:83:7D:AE:2C:87:9D:56:45:CE:25
  Signature Algorithm: ecdsa-with-SHA256
  Signature Value:
    30:46:02:21:00:8e:a3:91:48:0a:d5:8a:9d:a7:ee:08:ae:27:...
```

Satır satır: **Version 3**, uzantı alanlarını (X509v3 bloğu) kullanabildiğimizi gösterir. **Serial Number**, bu CA'nın çıkardığı sertifikalar arasında benzersizdir — bir sertifika iptal edildiğinde CRL'de (Bölüm 10) bu numara listelenir. **Issuer** (CN=CEN429 Lab Ara CA), bu sertifikayı **kimin imzaladığını** söyler — Bölüm 7'deki "Soru 1" in cevabı burada yazar. **Validity**, Bölüm 7'deki "Soru 2". **Subject** (CN=localhost), sertifikanın **kime ait olduğunu** iddia ettiği kimlik — ama bu iddiaya güvenmeden önce SAN'a bakmalısınız (aşağıda). **Subject Public Key Info**, bu sertifikanın **doğruladığı** açık anahtarın kendisi (256 bit P-256 eğrisi üzerinde bir nokta). **Basic Constraints: CA:FALSE**, Bölüm 7'deki "Soru 3"ün cevabı: bu sertifika başka sertifika imzalayamaz. **Subject Alternative Name**, Bölüm 7'deki "Soru 4"ün cevabı. **Authority Key Identifier**, bu sertifikayı imzalayan CA'nın açık anahtarının parmak izidir; istemci zinciri kurarken "issuer adı aynı olan ama farklı bir anahtarla imzalanmış sahte bir ara sertifika" karışıklığını bununla önler. En altta **Signature Value**, bütün bu alanların

(TBSCertificate) üzerinde, **CA'nın özel anahtarıyla** üretilmiş imzadır — Bölüm 5'te öğrendiğimiz imza mekanizmasının birebir kendisi.

Sayısal örnek: sertifika kaç bayt?

```
openssl x509 -in sunucu.crt -outform DER | wc -c
wc -c sunucu.crt
```

DER (binary) biçim genelde 500–900 bayt civarındadır (eğri tabanlı anahtarlar RSA'dan küçük olduğu için); PEM biçimi Base64 kodlaması yüzünden bundan yaklaşık %33 daha büyüktür (Base64, her 3 binary bayt 4 metin karakterine çevirir). Bir TLS el sıkışmasında sunucunun gönderdiği sertifika **zinciri** (uç + ara), bu boyutların toplamıdır; RSA-3072 sertifikaları kullanan bir zincirin ECC (P-256/Ed25519) zincirinden **kat kat büyük** olmasının nedeni budur — Bölüm 4'teki boyut karşılaştırmasının PKI'daki yansıması.



Sık yapılan hata: Subject 'teki CN'e güvenmek

Subject: CN=localhost alanı yalnız bir **iddiadır**; CA bu iddiayı doğruladığı için sertifikayı imzalamıştır, ama istemci **ad denetimini** bu alana değil SAN'a yapmalıdır (yukarıdaki uyarı kutusu). Eski kod örnekleri ve bazı öğretim materyalleri hâlâ CN'e bakan kod gösterir; bu RFC 6125 ile terk edilmiş bir yaklaşımdır.



Kural

Bir sertifikayı elle incelerken her zaman `openssl x509 -noout -text` (ya da `-ext subjectAltName`) kullanın ve **SAN, geçerlilik, basicConstraints, issuer/subject zinciri** dördlüsünü (Bölüm 7) birlikte okuyun; tek bir alana bakıp karar vermeyin.

Bölüm 7 sertifikanın **taşıdığı güveni** anlattı; bu bölüm o güvenin **hangi baytlarda** kodlandığını gösterdi. Sıradaki bölüm bu belgeyi CA rolüne geçip **elle üretiyoruz**.

9. OpenSSL ile sertifika zinciri kurmak

Aşağıdaki adımlar, kendi laboratuvarınızda üç katmanlı bir zincir kurar. Bütün dosyalar çalışma klasöründe oluşur; kök sertifikayı **hiçbir güven deposuna eklemeyin**.

Zincir doğrulamanın dört sorusu

+ ad denetimi (SAN) — beşi birden geçmeden kabul edilmez



Eksik ara sertifika zinciri kurdurmaz; fail-open ile “geçti” saymak en sık hatadır.

1. Kök CA (kendi kendini imzalar)

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out kok.key
openssl req -x509 -new -key kok.key -sha256 -days 3650 -out kok.crt \
  -subj "/CN=CEN429 Lab Kok CA" \
  -addext "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign"
```

2. Ara CA (kök imzalar, yol uzunluğu 0: altında başka CA olamaz)

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out ara.key
openssl req -new -key ara.key -out ara.csr -subj "/CN=CEN429 Lab Ara CA"
openssl x509 -req -in ara.csr -CA kok.crt -CAkey kok.key -CAcreateserial -days 1825 -sha256 \
  -out ara.crt -extfile <(printf
"basicConstraints=critical,CA:TRUE,pathlen:0\nkeyUsage=critical,keyCertSign,cRLSign")
```

3. Sunucu sertifikası (ara CA imzalar, SAN ile)

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out sunucu.key
openssl req -new -key sunucu.key -out sunucu.csr -subj "/CN=localhost"
openssl x509 -req -in sunucu.csr -CA ara.crt -CAkey ara.key -CAcreateserial -days 90 -sha256 \
  -out sunucu.crt -extfile <(printf
"basicConstraints=CA:FALSE\nkeyUsage=critical,digitalSignature\nextendedKeyUsage=serverAuth\nsubjectAltName=DNS:localhost,IP:127.0.0.1")
```

4. Doğrula ve incele

```
openssl verify -CAfile kok.crt -untrusted ara.crt sunucu.crt      # sunucu.crt: OK
openssl x509 -in sunucu.crt -noout -text | less                 # alanları inceleyin
openssl verify -CAfile kok.crt sunucu.crt                       # ara olmadan: HATA (zincir eksik)
```



Windows'ta

<(...) süreç yerine koyma sözdizimi Bash'e özgüdür. Windows PowerShell'de uzantıları bir dosyaya yazıp (ara.ext, sunucu.ext) -extfile ara.ext biçiminde verin; komutların geri kalanı aynıdır.

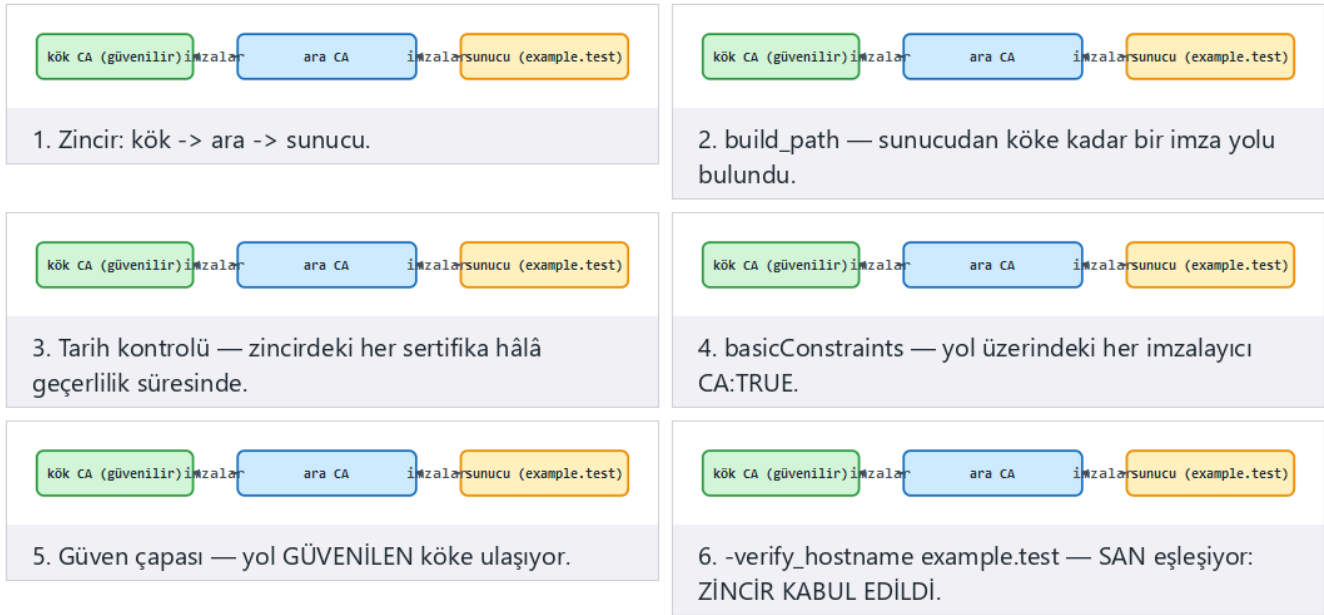
Son komutun hata vermesi öğreticidir: istemci zinciri ancak ara sertifika da sunulursa kurabilir. Sunucuların yalnız uç sertifikayı gönderip ara sertifikayı unutması, sahada sık görülen bir yapılandırma hatasıdır.

Sınıf etkinliği: zinciri bozmak

Her adımda `openssl verify` çıktısını not edin:

1. Sunucu sertifikasını **kök** ile imzalayın ama ara CA'yı zincirde bırakın. Ne değişir?
2. Ara CA'yı `CA:FALSE` ile üretip sunucu sertifikasını onunla imzalayın. Doğrulama ne der?
3. `-days 0` ile süresi hemen dolan bir sertifika üretin.
4. SAN'ı `DNS:baska.ornek` yapıp `openssl s_client -verify_hostname localhost` ile deneyin.

Dört sorunun (imza? köke ulaşıyor mu? süresi geçerli mi? ad uyuyor mu?) altısını — eksik ara, süresi dolmuş, yanlış kök, `CA:FALSE` imzalayıcı, yanlış ad ve tam zincir — aynı animasyonda deneyip 3. ve 4. senaryoyu kendiniz çözmeden önce fikir edinin:



Örnek seçicide **Uç durum: sertifikanın süresi dolmuş** ve **Uç durum: istenen ad SAN listesinde yok** yukarıdaki 3. ve 4. sorulara karşılık gelir.

İşlenmiş örnek: sınıf etkinliğinin iki senaryosunu baştan sona çözmek

Yukarıdaki dört senaryonun ilk ikisini birlikte çözelim; 3. ve 4. senaryoyu siz tamamlayacaksınız.

Senaryo 1 — Sunucu sertifikasını doğrudan kök ile imzalamak (ara CA'yı atlamak):

```
openssl x509 -req -in sunucu.csr -CA kok.crt -CAkey kok.key -CAcreateserial -days 90 -sha256 \
-out sunucu_kok.crt -extfile sunucu.ext
openssl verify -CAfile kok.crt -untrusted ara.crt sunucu_kok.crt
```

sunucu_kok.crt: OK

Şaşırtıcı gelebilir: ara.crt 'yi de verdiğimiz hâlde doğrulama **geçti**, çünkü OpenSSL zinciri sunucu_kok.crt 'den **doğrudan** kok.crt 'ye kurabildi; ara.crt bu yolda hiç kullanılmadı, sadece kullanılabilecek bir aday olarak havuzda durdu. Buradaki ders **kriptografik değil operasyoneldir**: matematiksel olarak kök CA, istediği her sertifikayı doğrudan imzalayabilir. Ama bunu **yapmamalıyız**, çünkü kök CA'nın özel anahtarı her gün kullanılan bir sistemde bulunursa (Bölüm 7'deki "neden ara CA" açıklaması), ele geçirilme riski artar ve bütün zincir çöker.

Senaryo 2 — Ara CA'yı CA:FALSE ile üretmek:

```
printf "basicConstraints=critical,CA:FALSE\nkeyUsage=critical,keyCertSign,cRLSign\n" > ara_yanlis.ext
openssl x509 -req -in ara.csr -CA kok.crt -CAkey kok.key -CAcreateserial -days 1825 -sha256 \
-out ara_yanlis.crt -extfile ara_yanlis.ext

openssl x509 -req -in sunucu.csr -CA ara_yanlis.crt -CAkey ara.key -CAcreateserial -days 90 -sha256 \
-out sunucu_yanlis.crt -extfile sunucu.ext

openssl verify -CAfile kok.crt -untrusted ara_yanlis.crt sunucu_yanlis.crt
```

```
CN=CEN429 Lab Ara CA
error 79 at 1 depth lookup: invalid CA certificate
error sunucu_yanlis.crt: verification failed
```

Bu kez doğrulama **reddedildi** — `ara_yanlis.crt` 'nin `basicConstraints` 'i `CA:FALSE` olduğu için, matematiksel imza doğru olsa bile OpenSSL bu sertifikanın **başka bir sertifika imzalama yetkisi olmadığını** görüyor ve zinciri "depth 1"de (aradaki katmanda) geçersiz sayıyor. Bu, Bölüm 8'deki `Basic Constraints` alanının **davranışı fiilen değiştiren** bir uzantı olduğunun kanıtıdır — süslü bir metin alanı değildir.

Senaryo 1 ile 2'yi karşılaştırın: ikisi de "yanlış" bir zincir kurar, ama biri (**kök ile doğrudan imzalama**) kriptografik olarak geçerlidir ve yalnız operasyonel bir hatadır; diğeri (**CA:FALSE ile imzalama**) protokol düzeyinde reddedilir. Bir değerlendirici PKI kurulumunuzu incelerken bu ayrımı tam olarak arar: "zincir teknik olarak çalışıyor mu" ve "zincir tasarım kurallarına uyuyor mu" iki ayrı sorudur.

10. Sertifika iptali: CRL, OCSP ve yanıt ilişirme (Tarif 10.10–10.12)

Bir sertifikanın özel anahtarı çalınırsa ya da sertifika hatalı çıkarılmışsa, geçerlilik süresi dolmadan **iptal** edilmesi gerekir. İstemcinin bunu öğrenmesinin üç yolu vardır:

Sertifika iptali: CRL · OCSP · zımbalama

"bu sertifika hâlâ geçerli mi?"



FAIL-OPEN TUZAĞI: "erişilemedi" = "geçti" sayma!

Yöntem	Nasıl?	Artı	Eksi
CRL (iptal listesi)	CA, iptal edilen seri numaralarının imzalı listesini düzenli yayımlar	Basit, çevrimdışı önbelleklenebilir	Liste büyür; güncelleme aralığında iptal fark edilmez
OCSP (RFC 2560, 1999)	İstemci CA'nın yanıtlayıcısına "bu seri numarası geçerli mi?" diye sorar	Anlık	Gecikme, gizlilik (CA hangi siteye gidildiğini öğrenir), yanıtlayıcı erişilemezse ne olacak?
OCSP yanıtını ilişirme (stapling)	Sunucu kendi sertifikasının taze OCSP yanıtını el sıkışmaya ekler	Gizlilik ve hız	Sunucunun yapılandırması gerekir

Laboratuvar CA'sıyla iptal ve CRL denetimi (özet)

```
# ara CA ile bir CRL üretmek için 'openssl ca' yapılandırması gerekir; sonra:
openssl ca -config ara.cnf -revoke sunucu.crt
openssl ca -config ara.cnf -gencrl -out ara.crl
openssl verify -crl_check -CAfile kok.crt -untrusted ara.crt -CRLfile ara.crl sunucu.crt #
"certificate revoked"
```



Yumuşak başarısızlık (soft-fail)

Tarayıcıların çoğu, OCSP yanıtlayıcısına ulaşamadığında sertifikayı **kabul eder**; aksi halde yanıtlayıcının kısa bir kesintisi bütün siteleri erişilemez kılar. Ama bu, araya giren bir saldırganın OCSP isteğini engelleyerek iptal denetimini atlatabileceği anlamına gelir: **3. haftadaki fail-open** örüntüsünün PKI'deki karşılığı. Kritik uygulamalarda "zımbalanmış yanıt zorunlu" (OCSP Must-Staple) ya da kısa ömürlü sertifikalar (günler, haftalar) bu sorunu azaltır; bu yüzden sertifika ömürleri giderek kısalmaktadır.

İşlenmiş örnek: bir sertifikayı iptal edip CRL ile doğrulamak (uçtan uca)

Yukarıdaki "özet" komutların tam hâlini, gerçekten çalıştırılmış olarak adım adım izleyelim. Önce `openssl ca`'nın ihtiyaç duyduğu minimal bir veritabanı ve yapılandırma dosyası kurulur (bu, gerçek bir CA'nın çok küçültülmüş hâlidir):

1. Minimal CA veritabanı kurmak

```
mkdir -p demoCA/newcerts
touch demoCA/index.txt # her cikarilan/iptal edilen sertifikanin kaydedildigi "defter"
echo 1000 > demoCA/crlnumber # bir sonraki CRL'in surum numarasi
echo 1000 > demoCA/serial # bir sonraki sertifikanin seri numarasi
```

ara.cnf

```
[ ca ]
default_ca = ara_ca

[ ara_ca ]
dir = demoCA
database = $dir/index.txt
new_certs_dir = $dir/newcerts
certificate = ara.crt
private_key = ara.key
serial = $dir/serial
crlnumber = $dir/crlnumber
default_md = sha256
default_days = 90
default_crl_days = 30
policy = policy_any
email_in_dn = no

[ policy_any ]
commonName = supplied
```

2. İptal et, CRL üret, doğrula (gerçek çıktı)

```
openssl ca -config ara.cnf -revoke sunucu.crt
```

```
Using configuration from ara.cnf
Adding Entry with serial number 35E2F68D...76E7 to DB for /CN=localhost
Revoking Certificate 35E2F68D...76E7.
Database updated
```

```
openssl ca -config ara.cnf -gencrl -out ara.crl
openssl verify -crl_check -CAfile kok.crt -untrusted ara.crt -CRLfile ara.crl sunucu.crt
```

```
CN=localhost
error 23 at 0 depth lookup: certificate revoked
error sunucu.crt: verification failed
```

Adım adım ne oldu: (1) `-revoke`, sertifikanın **seri numarasını** (Bölüm 8'deki `Serial Number` alanı — burada `35E2F68D...76E7`) `index.txt` defterine "iptal edildi" olarak işledi; sertifikanın kendisi hiçbir şekilde değişmedi, silinmedi — yalnız bir **kayıt** eklendi. (2) `-gencrl`, bu defterdeki bütün iptal kayıtlarını toplayıp **ara CA'nın kendi özel anahtarıyla imzaladığı** bir liste (`ara.crl`) üretti — CRL'in kendisi de bir imzalı belgedir, sahteciliğe karşı Bölüm 5'teki mekanizmayla korunur. (3) `-crl_check` ile doğrulama artık yalnız zinciri değil, CRL'i de kontrol etti ve seri numarasının listede olduğunu görüp **"certificate revoked"** hatasıyla reddetti — sertifikanın geçerlilik tarihi hâlâ dolmamış olsa bile.

Bu, Alıştırma 6'nın tam çözümüdür; kendi laboratuvarınızda aynı adımları tekrarlayıp `index.txt` dosyasının içeriğini (`cat demoCA/index.txt`) inceleyin — her satır bir sertifikanın kaydını taşır (durum, tarih, seri numarası, konu adı).

CRL'in neden **bayat** olabileceğini (kendi `nextUpdate` 'i geçmişse, "iyi" demek yerine "bayat" demesi gerektiğini) ve OCSP'nin neden her zaman **anlık** kaldığını, aynı sertifika üzerinde iki yöntemi yan yana çalıştırarak görün:

sertifika, seri #4711 yayımlandı, iptal edilmedi 1. CA durumu: seri #4711 yayımlandı, iptal EDİLMEDİ.	sertifika, seri #4711 yayımlandı, iptal edilmedi OCSP: <code>ocsp_query(#4711) = "good"</code> 2. <code>ocsp_query(4711)</code> — CANLI sorgu, şu anki bilgiye göre sonuç: "good".
sertifika, seri #4711 yayımlandı, iptal edilmedi OCSP: <code>ocsp_query(#4711) = "good"</code> CRL: <code>crl_check(#4711) = "good"</code> 3. <code>crl_check(4711, crl, at_time)</code> — CRL hâlâ taze, sonuç: "good".	

Zor: sertifika iptal edilmiş ile Uç durum: CRL bayat — iptal görülüyor örneklerini karşılaştırıp CRL'in önbelleklenebilirlik kazancının bedelini görün.

Sayısal örnek: CRL ne kadar büyür?

Bir CA yılda 10.000 sertifika çıkarıyor ve bunların %1'i (100 tanesi) iptal ediliyorsa, CRL her girişte ~40-60 bayt (seri numarası + iptal tarihi + sebep kodu) taşıdığından, CRL boyutu ~4-6 KB olur — küçük görünür. Ama büyük bir kamu CA'sı

(milyonlarca sertifika, yüz binlerce iptal) için CRL **onlarca megabayta** ulaşabilir; her istemcinin bunu periyodik indirmesi pratik değildir. OCSP bu ölçeklenme sorununu "listenin tamamını indirme" yerine "tek bir soru sor" ile çözer — ama bu kez de yukarıdaki gizlilik ve kullanılabilirlik ödünleşimi ortaya çıkar.



Sık yapılan hata: iptal denetimini yalnız geliştirme ortamında test edip üretime öyle almak

`-crl_check` (ya da OCSP eşdeğeri) açık olmadan yazılan bir istemci, geçerlilik tarihi dolmamış ama **iptal edilmiş** bir sertifikayı sorunsuz kabul eder. Test ortamında CRL/OCSP sunucusu genelde kurulu olmadığından bu hata sık atlanır; üretime "iptal denetimi kapalı" biçimde taşınır. Kural: iptal denetimini **açık bir yapılandırma seçeneği** yapın ve dağıtım kontrol listesinde bunun açık olduğunu doğrulayın.



Kural

Sertifika ömrü ne kadar kısaysa, iptal mekanizmasına o kadar az ihtiyaç duyarsınız — bu yüzden endüstri (Let's Encrypt ve benzerleri) sertifika ömürlerini kısaltma yönünde ilerliyor. Uzun ömürlü sertifikalar kullanıyorsanız CRL ya da OCSP yanıtını iliştiirmeyi **mutlaka** etkinleştirin ve test edin.

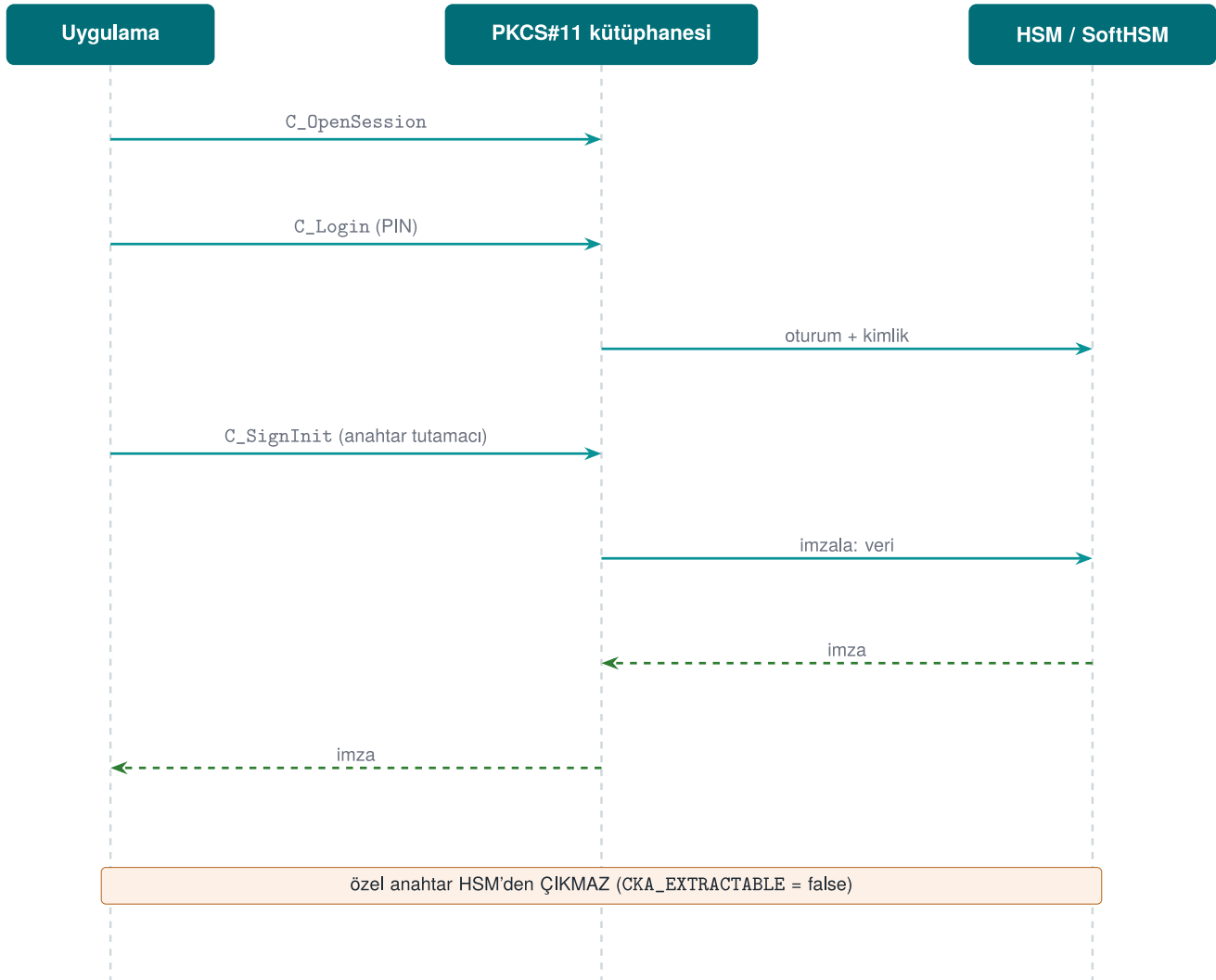
Bölüm 9'da bir zincir **kurmaya**, bu bölümde bir zincirdeki bir halkayı **geçersiz kılmaya** öğrendik. Sıradaki bölüm, bu zincirin en değerli parçasını — özel anahtarların kendisini — nasıl koruduğumuzu işliyor.

11. Anahtarı donanımda saklamak: HSM, PKCS#11 ve SoftHSM

Özel anahtarın çalınması PKI'nin bütün güvencesini yok eder. Bu yüzden değerli anahtarlar (CA anahtarları, ödeme anahtarları, kod imzalama anahtarları) **donanım güvenlik modüllerinde** (HSM) üretilir ve oradan hiç çıkmaz. Uygulama anahtarı hiç görmez; HSM'e "şu anahtarla şunu imzala" der.

PKCS#11 ile imzalama: anahtar hiç dışarı çıkmaz

uygulama anahtarı değil, anahtarın TUTAMACINI görür



Anahtar çalınmaz; ama HSM'e erişimi olan imzalatabilir — erişim denetimi yine şarttır.

Kavram	Açıklama
HSM	Kurcalamaya dayanıklı, sertifikalı (FIPS 140-3, Ortak Kriterler) donanım; anahtarı dışa aktarmaz
PKCS#11 (Cryptoki)	HSM'lere, akıllı kartlara ve kriptografik belirteçlere erişim için standart C API'si
Nesne ve öznitelikler	Anahtarlar <code>CKA_SENSITIVE</code> , <code>CKA_EXTRACTABLE=false</code> gibi özniteliklerle "dışa aktarılamaz" işaretlenir
Oturum ve PIN	Uygulama bir yuvaya (slot) oturum açar ve kullanıcı PIN'iyle kimlik doğrular
SoftHSM	PKCS#11 arayüzünü yazılımla sunan açık kaynak bir HSM benzetimi; geliştirme ve test için

SoftHSM ile bir anahtar üretmek ve listelemek (WSL/Linux)

```
export SOFTHSM2_CONF=$PWD/softhsm2.conf
mkdir -p belirteciler && echo "directories.tokendir = $PWD/belirteciler" > softhsm2.conf
softhsm2-util --init-token --free --label "cen429" --pin 1234 --so-pin 5678
pkcs11-tool --module /usr/lib/softhsm/libsofthsm2.so --login --pin 1234 \
--keypairgen --key-type EC:prime256v1 --label imza-anahtari
pkcs11-tool --module /usr/lib/softhsm/libsofthsm2.so --login --pin 1234 --list-objects
```

**SoftHSM bir HSM değildir**

SoftHSM anahtarları diskte, bir dosyada saklar; donanım koruması yoktur. Değeri, uygulamanızı **PKCS#11 arayüzüne göre** yazmanızı sağlamasıdır: aynı kod, üretimde gerçek bir HSM'e ya da akıllı karta yalnız modül yolu değiştirilerek bağlanır. **11. haftada** yazılım tabanlı güvenlik modüllerine bu açıdan yeniden bakacağız.

**Sahada nasıl uygulanır?**

Ödeme sistemlerinde anahtar hiyerarşisinin tepesindeki anahtarlar (kart çıkaran kurumun ana anahtarları, cihaz anahtarlarını üreten anahtarlar) sunucu tarafında HSM'lerde durur; mobil ödeme platformunun arka ucunda HSM, ayrı bir bileşen olarak mimari şemada yer alır (**1. haftadaki örnek mimari**). Telefona inen anahtarlar sunucuda HSM ile üretilir, cihaza bağlanarak şifrelenir ve kütüphanenin kendisi hesap anahtarı **üretmez**. Kılavuz, kullandığı her algoritmayı dayandığı standartla (NIST FIPS, ISO/IEC 9797-1, EMV belirtilimleri) birlikte listeleyen bir algoritma envanteri içerir.

Uçtan uca: bir imzalama isteği HSM içinde nasıl işlenir?

"Anahtar hiç çıkmaz" cümlesini somutlaştıralım. **PKCS#11** standardı (OASIS Cryptoki v3), uygulamanın bir HSM'e **asla ham anahtar baytı göndermeden ya da almadan** imza attırmasını şu adımlarla tanımlar:

1. **Oturum aç:** Uygulama `C_OpenSession` ile HSM'deki bir "yuvaya" (slot) bağlanır.
2. **Kimlik doğrula:** `C_Login` ile PIN gönderilir; PIN **yanlışsa** HSM belirli bir deneme sayısından sonra kendini kilitler (kaba kuvvete karşı donanım düzeyinde koruma — yazılımda unutulması kolay bir önlem).
3. **Anahtarı bul:** `C_FindObjectsInit` / `C_FindObjects` ile önceden üretilmiş anahtar nesnesinin **tutamacı** (handle — anahtarın kendisi değil, HSM içindeki bir referans numarası) alınır.
4. **İmzala:** `C_SignInit` + `C_Sign(tutamaç, veri, ...)` çağrısı, veriyi HSM'e gönderir; HSM veriyi **kendi içinde** anahtarla işler ve yalnız **imzayı** döndürür. Uygulama kodunun hiçbir noktasında özel anahtarın baytları belleğe **hiç girmez**.
5. **Oturumu kapat:** `C_Logout` / `C_CloseSession`.

PKCS#11 C_Sign çağrısının biçimi (OASIS Cryptoki v3, referans)

```
CK_RV C_Sign(
    CK_SESSION_HANDLE hSession, /* acik oturum */
    CK_BYTE_PTR       pData,     /* imzalanacak veri */
    CK_ULONG          ulDataLen,
    CK_BYTE_PTR       pSignature, /* HSM'in doldurdugu cikti tamponu: yalnız imza, anahtar degil */
    CK_ULONG_PTR      pulSignatureLen
);
```

Bu akışı normal bir OpenSSL çağrısıyla (`EVP_DigestSign`, Bölüm 5) karşılaştırın: ikisinin de **arayüzü** aynı fikri taşır ("veriyi ver, imzayı al"), ama OpenSSL'de anahtar (dosyadan okunmuşsa) **süreç belleğindedir**; **PKCS#11**'de anahtar **hiçbir zaman** uygulamanın adres alanına girmez. Bölüm 5'teki `imza_dogrula` fonksiyonunu bir HSM'e taşımak, yalnız imzalama tarafını `C_Sign` çağrısına yönlendirmek anlamına gelir — **doğrulama** tarafı zaten yalnız açık anahtar kullandığı için HSM'e ihtiyaç duymaz.

Aynı fikri küçük bir simülasyonda görün: `generate_keypair()` yalnız bir **tutamaç** döndürür, `sign(tutamaç, veri)` imzayı HSM'in İÇİNDE hesaplar, bilinmeyen ya da yok edilmiş bir tutamaçla imzalama reddedilir:



Zor: imza bir kulpla, doğrulama BAŞKA kulpla ile Uç durum: hiç üretilmemiş kulp örneklerini deneyin.

Neden `CKA_EXTRACTABLE` en kritik öznitelik?

Bir **PKCS#11** anahtar nesnesi üretilirken (`C_GenerateKeyPair`), `CKA_EXTRACTABLE=false` ve `CKA_SENSITIVE=true` öznitelikleri **anahtar üretilirken** verilir ve sonradan **değiştirilemez**. Bu iki bayrak, `C_WrapKey` gibi bir fonksiyon çağrılrsa bile HSM'in anahtarı düz metin olarak dışarı vermesini **donanım düzeyinde** engeller.

⚡ Sık yapılan hata: varsayılan özniteliklerle anahtar üretmek

Bazı **PKCS#11** kütüphaneleri ve araçları, açıkça belirtilmezse `CKA_EXTRACTABLE` 'ı **true** bırakır (uyumluluk için). Bu durumda anahtar teknik olarak bir HSM'de üretilse bile, doğru izinlere sahip biri onu **dışa aktarabilir** — "HSM kullanıyoruz" güvencesi boş bir cümleye döner. Kod incelemesinde `C_GenerateKeyPair` çağrısının öznitelik şablonunda `CKA_EXTRACTABLE = CK_FALSE` 'ın **açıkça** verildiğini arayın.

✓ Kural

Değerli bir anahtar (CA kök anahtarı, kod imzalama anahtarı, ödeme anahtarı) üretilirken

`CKA_EXTRACTABLE=false` ve `CKA_SENSITIVE=true` açıkça ayarlanmalı ve üretimden hemen sonra `pkcs11-tool --list-objects` (ya da eşdeğeri) ile bu özneliklerin gerçekten uygulandığı **doğrulanmalıdır** — varsayıla güvenmeyin.

Yazılım tabanlı güvenlik modüllerine (mobil cihazlardaki Secure Enclave/TEE benzeri yapılar) **11. haftada** bu bölümün **PKCS#11** mantığıyla yeniden bakacağız; burada öğrendiğiniz "tutamaç üzerinden çalış, ham anahtar asla taşıma" prensibi oradaki bütün tartışmanın temelidir. Sıradaki ve son teknik bölüm, bugün güvenli saydığımız RSA/ECC'nin gelecekte neden değişmesi gerektiğine bakıyor.

12. Kuantum sonrası kriptografiye bakış

Yeterince büyük bir kuantum bilgisayar, RSA'nın ve eliptik eğrilerin dayandığı matematiksel problemleri (çarpanlara ayırma, ayrık logaritma) verimli biçimde çözebilir. Böyle bir bilgisayar henüz yoktur; ama bugün kaydedilen şifreli trafiğin yıllar sonra çözülebileceği ("şimdi topla, sonra çöz") düşüncesi, geçişi bugünden gündeme getirir.

Kuantum sonrası: neyi değiştiriyor, neyi değiştirmiyor?

NIST seçimi: ML-KEM (Kyber) ve ML-DSA (Dilithium)

TEHDİT ALTINDA
RSA, DH, ECDH, ECDSA
"şimdi topla, sonra çöz"
bugünkü trafik ileride çözülebilir
geçiş bugün planlanır

BÜYÜK ÖLÇÜDE GÜVENLİ
AES-256 (anahtar boyu yeter)
SHA-384 / SHA-3
simetrik dünya ayakta
yalnız anahtar boyu artar

Uzun ömürlü sırlarınız varsa hibrit anahtar değişimine geçiş bugünün kararıdır.

Durum	Ne değişiyor?
Simetrik şifreler ve özetler	Etkisi sınırlı: AES-256 ve SHA-384/512 yeterli sayılır
Anahtar anlaşması	NIST 2024'te ML-KEM 'i (FIPS 203) standartlaştırdı; TLS'te X25519 ile birlikte karma kullanım yaygınlaşıyor
İmza	ML-DSA (FIPS 204) ve SLH-DSA (FIPS 205) standartları yayımlandı
Takvim	NIST'in geçiş planı RSA ve eliptik eğri algoritmalarını 2030'dan sonra aşamalı olarak bırakmayı öngörür

Bu dersin projesi için pratik sonucu tek cümledir: **kripto çevikliği** (crypto agility). Algoritma adlarını kodun her yerine gömmeyin; anahtar sürümü ve algoritma kimliği taşıyan biçimler (**3. haftadaki anahtar sürümü alanı**) kullanın ki algoritma değiştiğinde yalnız yapılandırma ve yeniden şifreleme gereksin.

İşlenmiş örnek: kuantum sonrası anahtarları bugünden üretmek

OpenSSL 3.5, standartlaşan kuantum sonrası algoritmaları **doğrudan** destekler; bunları bugün, kendi makinenizde üretip boyutlarını klasik algoritmalarla karşılaştırabilirsiniz:

ML-KEM-768 (anahtar anlaşması) ve ML-DSA-65 (imza) — gerçek çıktı

```
openssl genpkey -algorithm ML-KEM-768 -out mlkem.pem
openssl pkey -in mlkem.pem -pubout -out mlkem_pub.pem
wc -c mlkem_pub.pem

openssl genpkey -algorithm ML-DSA-65 -out mldsa.pem
openssl pkey -in mldsa.pem -pubout -out mldsa_pub.pem
echo hello > belge.txt
openssl pkeyutl -sign -inkey mldsa.pem -rawin -in belge.txt -out belge.mldsa.sig
wc -c mldsa_pub.pem belge.mldsa.sig
openssl pkeyutl -verify -pubin -inkey mldsa_pub.pem -rawin -in belge.txt -sigfile belge.mldsa.sig
```

```
1714 mlkem_pub.pem
2770 mldsa_pub.pem
3309 belge.mldsa.sig
Signature Verified Successfully
```

Bölüm 4'teki tabloyla karşılaştırın:

Algoritma	Açık anahtar (PEM, bayt)	İmza (bayt)	Güvenlik dayanağı
Ed25519 (klasik)	116	64	Eliptik eğri ayrık logaritma
RSA-3072 (klasik)	636	384	Çarpanlara ayırma
ML-DSA-65 (PQC)	2.770	3.309	Kafes tabanlı problem (Module-LWE)
ML-KEM-768 (PQC, anahtar anlaşması)	1.714	—	Kafes tabanlı problem (Module-LWE)

Fark çarpıcıdır: ML-DSA-65'in açık anahtarı Ed25519'unkinden **~24 kat**, imzası **~52 kat** büyüktür. Bu, PQC'ye geçişin yalnız "algoritma adını değiştirmek" olmadığını gösterir — TLS el sıkışması, sertifika boyutu, ağ paket sayısı gibi pek çok yer bu büyümeyi hesaba katmalıdır. NIST'in TLS için önerdiği **karma** (hybrid) yaklaşım (`X25519MLKEM768` gibi, Bölüm 6'daki X25519 anlaşmasının PQC ile birlikte çalıştırılan hâli) tam olarak bu geçiş maliyetini yönetmek içindir: klasik ve PQC anahtarları **aynı anda** kullanılır; klasik taraf kırılırsa PQC, PQC tarafı (henüz kanıtlanmamış yeni matematik) kırılırsa klasik taraf korumayı sürdürür.

Neden kafes problemleri? Kısa bir sezgi

RSA'nın güvenliği "büyük bir sayıyı çarpanlarına ayırmanın zorluğuna", ECC'nin güvenliği "bir eğri üzerinde ayrık logaritma almanın zorluğuna" dayanır — Shor algoritması, yeterince büyük bir kuantum bilgisayarda **her ikisini de** verimli çözer. ML-KEM ve ML-DSA, bunun yerine **kafes** (lattice) problemlerine dayanır: çok boyutlu bir uzayda, düzenli aralıklarla dizilmiş noktalardan (kafes) oluşan bir yapıda, "verilen bir noktaya en yakın kafes noktasını bulmak" ya da "bir denklem sistemini küçük hata terimleriyle çözmek" (Learning With Errors, LWE) yüksek boyutlarda hesaplama açısından çok zordur ve

bugüne kadar ne klasik ne kuantum bilgisayarlar için verimli bir çözüm bulunmuştur. "Bugüne kadar bulunamamış olması", "asla bulunamayacağı" anlamına gelmez — bu yüzden karma yaklaşım öneriliyor.



Sık yapılan hata: 'kuantum bilgisayar yok, acelesi yok' demek

Bugün RSA/ECC ile şifrelenip kaydedilen (ör. bir istihbarat kaydı, uzun süreli bir tıbbi kayıt, 20-30 yıl gizli kalması gereken bir devlet belgesi) trafik, "şimdi topla, sonra çöz" (harvest now, decrypt later) stratejisiyle bugün kaydedilip kuantum bilgisayar olgunlaştığında çözülebilir. Gizlilik penceresi 10-20 yıl olan bir sistem için geçiş kararı **bugün** alınmalıdır, kuantum bilgisayar var olduğunda değil.



Kural

Yeni bir protokol tasarlıyorsanız, mümkünse şimdiden **karma** (klasik + PQC) anahtar anlaşımını destekleyin ya da en azından Bölüm 1'deki kriptto çevikliği ilkesiyle (algoritma kimliği/sürümü taşıyan biçimler) ileride PQC'ye geçişi **yeniden yazmadan** yapılabilir hâle getirin.

Bu bölüm, Bölüm 1'de çizdiğimiz haritanın **son durağıydı**: hangi algoritma, hangi problem, ne kadar güvenli, ne zamana kadar. 13. bölümden itibaren bu bilgiyi kendi projenizin algoritma envanterine döküyorsunuz.

13. Dönem projesi: bu hafta

Final kontrolünde beklenen **S8** ve **S11** bölümlerinin taslağını yazın:

- ✓ **S8 — Algoritma envanteri**: projenizde kullanılan her algoritma için amaç, anahtar uzunluğu, kip, dayandığı standart ve kütüphane. Tabloda eskimiş bir algoritma varsa gerekçesi ve geçiş planı.
- ✓ **S8 — Anahtar yaşam döngüsü tablosu**: her anahtar için üretildiği yer, saklandığı yer, kriptto-periyot, yenileme ve imha (3. hafta).
- ✓ **S11 — Güvenli iletişim**: TLS sürümü ve doğrulama; sabitleme varsa SPKI ve yedek pin; mesaj düzeyi koruma.
- ✓ Projenizde bir güncelleme ya da yapılandırma dosyası varsa, **imza doğrulamasını** ekleyin (Ed25519 önerilir) ve sürüm geri almayı reddedin.
- ✓ İsteğe bağlı: projenizin test sertifikaları için bu haftanın kök → ara → uç zincirini kullanın; kök anahtarı depoya **koymayın**.

İşlenmiş örnek: S8 tablosunu bir örnek bileşen için doldurmak

Boş bir tablo karşısında ne yazılacağı belirsiz kalabilir; örnek bir proje bileşeni ("kullanıcı oturum belirteci üretimi") için S8'in nasıl doldurulacağını görelim — kendi projenizde satırları kendi bileşenlerinizle değiştirin:

İş	Algoritma	Anahtar uzunluğu	Kip/Dolgu	Kütüphane	Dayandığı standart	Not
Oturum verisi şifreleme	AES-256	256 bit	GCM (AEAD)	OpenSSL EVP 3.x	NIST SP 800-38D	Bölüm 2
Oturum bütünlüğü	— (GCM etiketi)	—	—	OpenSSL EVP	SP 800-38D	AEAD içinde, ayrı MAC yok
Anahtar anlaşması	X25519	128 bit düzeyi	ECDH	OpenSSL EVP	RFC 7748	Bölüm 6
Sunucu kimlik doğrulama	ECDSA P-256	128 bit düzeyi	—	OpenSSL/TLS	FIPS 186-5	TLS sertifikası, Bölüm 7–9
Güncelleme paketi imzası	Ed25519	128 bit düzeyi	—	OpenSSL EVP	RFC 8032	Bölüm 5, sürüm alanı imzalı

Anahtar yaşam döngüsü tablosu (3. haftadaki anahtar yönetimi konusunun bu haftaki karşılığı) aynı bileşen için:

Anahtar	Üretildiği yer	Saklandığı yer	Kripto-periyot	Yenileme	İmha
Oturum anahtarı (AES-256)	Sunucu, her oturumda	Yalnız bellekte	Tek oturum	Her oturumda yeni	Oturum kapanınca bellekten silinir
TLS sunucu özel anahtarı	Sunucu, HSM/SoftHSM	HSM (Bölüm 11)	90 gün (sertifika ömrü)	Otomatik yenileme (ACME)	HSM'den <code>CKA_EXTRACTABLE=false</code> ile hiç çıkmaz
Güncelleme imzalama anahtarı	Çevrimdışı, tören ile	HSM, çevrimdışı yedek	Ürün ömrü boyunca	Yılda bir gözden geçirme	Kurum politikasına göre

Bu iki tablo, bu haftanın 1–12. bölümlerinde gördüğünüz her kavramı ("hangi kip", "hangi standart", "nerede saklanır", "ne zaman yenilenir") tek bir yerde toplar — final kontrolünde değerlendirici tam olarak bu iki tabloyu arar.



Tabloyu kendi projenize uyarlarken

Her satırı kendi kod tabanınızdaki **gerçek** bir çağrıya (dosya adı ve satır numarası dahi verebilirsiniz) bağlayın; "AES kullanıyoruz" gibi genel bir cümle yerine "auth.c:142 satırında `EVP_EncryptInit_ex` ile AES-256-GCM" gibi izlenebilir bir referans yazın. Bölüm 1'deki "algoritma adı tek başına yeterli değildir" kuralının final kontrolündeki karşılığı budur.

14. Kendi başına çalış

? Alıştırma 1 — Kolay: HMAC ve uzunluk uzatma



Aynı mesaj ve anahtar için `openssl dgst -sha256 -mac HMAC` ile HMAC ve `printf 'anahtar+mesaj' | openssl dgst -sha256` ile düz özet hesaplayın. Düz $H(K \parallel m)$ yapısının neden güvensiz olduğunu uzunluk uzatma kavramıyla açıklayın.

? Alıştırma 2 — Kolay: RSA ve Ed25519 karşılaştırması



3072 bitlik bir RSA anahtarı ve bir Ed25519 anahtarı üretin. Açık anahtar dosyalarının, imzaların boyutlarını ve 100 imzalama işleminin süresini karşılaştırın (`openssl speed rsa3072 ed25519` de kullanılabilir).

? Alıştırma 3 — Orta: İmza doğrulamasının dönüş değeri



"Dijital imza" bölümündeki `imza_dogrula` fonksiyonunu çalıştırın. Önce doğru bir imzayla, sonra bir baytı değiştirilmiş imzayla, sonra hatalı bir anahtar nesnesiyle deneyin. `if (sonuc)` ile `if (sonuc == 1)` arasındaki farkın hangi durumda güvenlik açığına dönüştüğünü gösterin.

? Alıştırma 4 — Orta: Zinciri bozmak



"Sınıf etkinliği" bölümündeki dört senaryoyu uygulayın ve her birinde `openssl verify` ile `openssl s_client` hata iletilerini bir tabloya yazın.

? Alıştırma 5 — Orta: TLS sunucusu ve zincir



Oluşturduğunuz sunucu sertifikasıyla yerel bir TLS sunucusu başlatın (`openssl s_server -cert sunucu.crt -key sunucu.key -cert_chain ara.crt -accept 8443`) ve `openssl s_client -connect localhost:8443 -CAfile kok.crt -verify_return_error` ile bağlanın. Sunucu ara sertifikayı göndermezse ne olur?

? Alıştırma 6 — Orta: İptal



Laboratuvar ara CA'nız için bir `openssl ca` yapılandırması hazırlayın, sunucu sertifikasını iptal edin, CRL üretin ve `openssl verify -crl_check` ile iptalin fark edildiğini gösterin.

? Alıştırma 7 — Zor: Yeniden oynatmaya dayanıklı ileti



İki program yazın: biri `sayaç || veri` biçiminde iletiyi AES-GCM ile şifreleyip (sayaç AAD olsun) gönderir, diğeri alır ve daha önce görülen ya da daha küçük bir sayacı reddeder. Kaydedilen bir iletiyi yeniden gönderip reddedildiğini gösterin.

? Alıştırma 8 — Zor: PKCS#11 ile imza

SoftHSM'de ürettiğiniz anahtarla `pkcs11-tool --sign` kullanarak bir dosyayı imzalayın ve imzayı OpenSSL ile açık anahtardan doğrulayın. Özel anahtarı SoftHSM'den dışa aktarmaya çalışın; `CKA_EXTRACTABLE` özneliği bunu nasıl engelliyor?

15. Kendini sinama

? 1. 128 bit güvenlik düzeyi için RSA ve eliptik eğride anahtar uzunluğu nedir?

RSA 3072 bit, eliptik eğri 256 bit (P-256, Ed25519/X25519).

? 2. Dolgu kâhini saldırısı hangi koşulda mümkündür? İki savunma verin.

Çözücü "dolgu geçersiz" ile başka hataları (farklı ileti ya da süreyle) ayırt edilebilir biçimde bildiriyorsa. AEAD kullanmak; CBC zorunluysa önce şifrele, sonra MAC ve MAC'i dolgudan önce doğrulamak, bütün hataları aynı biçimde döndürmek.

? 3. Neden HMAC kullanılır da $H(K \parallel m)$ kullanılmaz?

Merkle–Damgård yapısındaki özetlerde $H(K \parallel m)$ uzunluk uzatma saldırısına açıktır; HMAC'in iç içe yapısı bunu önler.

? 4. Önce şifrele, sonra MAC neden doğru sıradır?

MAC şifreli metin üzerinden hesaplanır ve çözmeden önce doğrulanır; geçersiz ileti hiç çözülmez, dolgu kâhini ve düz metin sızıntısı oluşmaz.

? 5. MAC yeniden oynatmayı neden tek başına önlemez? Nasıl önlenir?

MAC iletinin değiştirilmediğini kanıtlar, yeni olduğunu kanıtlamaz. Sayaç, zaman damgası ya da sunucunun verdiği tek kullanımlık değer MAC'e katılır ve alıcı tekrarları reddeder.

? 6. RSA şifrelemede ve imzada hangi dolgu şemaları önerilir?

Şifrelemede OAEP (SHA-256), imzada PSS.

? 7. ECDSA'da aynı rastgele değerin iki imzada kullanılması neden felakettir?

İki imzadan özel anahtar hesaplanabilir. Ed25519 ya da deterministik ECDSA (RFC 6979) bu riski ortadan kaldırır.

? 8. `EVP_DigestVerify` sonucunu `if (sonuc)` ile denetlemek neden hatalıdır?

Negatif hata değerleri de doğru sayılır; hata durumunda imza geçerli kabul edilir. Denetim `== 1` olmalıdır.

? 9. Kimliksiz Diffie–Hellman neden araya girmeye açıktır? TLS 1.3 bunu nasıl çözer?

DH kiminle anlaşıldığını söylemez; aradaki biri iki tarafla ayrı ayrı DH yapabilir. TLS 1.3'te sunucu, geçici DH değerlerini de içeren el sıkışma özetini sertifikasındaki anahtarla imzalar; istemci sertifikayı doğrular.

? 10. Kök CA neden çevrimdışı tutulur ve ara CA neden vardır?

Kök anahtarın ele geçirilmesi bütün zinciri çökertir ve kökü güven depolarından çıkarmak yıllar sürer. Günlük işi ara CA yapar; ele geçirilirse yalnız o iptal edilir.

? 11. Ad denetimi neden CN'ye değil SAN'a yapılır?

Güncel standartlar ve istemciler yalnız SAN uzantısını dikkate alır; CN'ye bakmak eski ve belirsiz bir yöntemdir.

? 12. OCSP'de 'yumuşak başarısızlık' nedir ve neden tehlikelidir?

Yanıtlayıcıya ulaşılamazsa sertifikanın kabul edilmesidir. Araya giren saldırgan OCSP isteğini engelleyerek iptal denetimini atlatabilir (fail-open). Yanıt iletme zorunluluğu ya da kısa ömürlü sertifikalar bunu azaltır.

? 13. SoftHSM'in gerçek bir HSM'den farkı nedir? Neden yine de kullanılır?

Anahtarları diskte saklar, donanım koruması yoktur. Uygulamayı PKCS#11 arayüzüne göre yazmayı sağlar; üretimde aynı kod gerçek HSM'e bağlanır.

? 14. Kripto çevikliği nedir ve kuantum sonrası geçişle ilişkisi nedir?

Algoritmaları kodun her yerine gömmeden, sürüm ve algoritma kimliği taşıyan biçimlerle değiştirilebilir tutmak. ML-KEM ve ML-DSA gibi yeni algoritmalara geçiş, yalnız yapılandırma ve yeniden şifrelemeyle yapılabilir.

? 15. 128 bit güvenlik düzeyinde bir AES anahtarını kaba kuvvetle kırmanın, saniyede 10^{18} işlem yapabilen varsayımsal bir bilgisayarla bile neden pratik olmadığını açıklayın. ✓

2^{128} işlem gerekir ($\approx 3,40 \times 10^{38}$). Bu hızla bile $\approx 1,08 \times 10^{13}$ yıl sürer — evrenin yaşının ($\approx 1,38 \times 10^{10}$ yıl) yaklaşık 780 katı. Bu yüzden AES-128 bugün ve öngörülebilir gelecekte kaba kuvvete karşı güvenli sayılır.

? 16. PKCS#7 dolgusunda 13 baytlık bir mesaj 16 baytlık bir blokta şifrelenirse, dolgu kaç bayttır ve değeri nedir? ✓

3 bayt; her biri 03 değerinde ($16 - 13 = 3$, eksik bayt sayısı kadar, o sayı değerinde bayt eklenir).

? 17. ECB kipinde tekrar eden düz metin blokları neden tekrar eden şifreli metin blokları üretir? CBC bunu nasıl önler? ✓

ECB her bloğu bağımsız ve aynı anahtarla şifreler; aynı girdi her zaman aynı çıktıyı verir. CBC her bloğu bir öncekiyle (ilk blokta IV ile) XOR'layarak zincirler; bu yüzden aynı düz metin blokları farklı önceki bloklara bağlı olarak farklı şifreli metin üretir.

? 18. HMAC'te tek bir karakterlik değişiklik etiketin ne kadarını değiştirir? Bu özelliğin adı nedir? ✓

Yaklaşık yarısını. Buna çığ etkisi (avalanche effect) denir.

? 19. MAC/imza karşılaştırmasında `memcmp` yerine neden sabit zamanlı bir fonksiyon kullanılır? ✓

`memcmp` ilk farklı baytta durur; bu, karşılaştırma süresini doğru baytların sayısıyla orantılı hâle getirir. Yeterince çok deneme yapabilen bir saldırgan, yanıt süresindeki farktan doğru değeri bayt bayt keşfedebilir (zamanlama saldırısı). Sabit zamanlı fonksiyon (`CRYPTO_memcmp` gibi) her zaman bütün baytları gezer.

? 20. Aynı düz metni iki kez RSA-OAEP ile şifrelediğinizde neden farklı şifreli metinler elde edersiniz? ✓

OAEP her şifrelemede dolgu içine rastgele bir değer karıştırır. Ham/dolgusuz RSA deterministiktir ($c = m^e \bmod n$) ve aynı girdi için hep aynı çıktıyı verir; bu yüzden ham RSA doğrudan şifreleme için kullanılmaz.

? 21. X25519 ile Alice ve Bob'un aynı ortak sırra ulaştığını nasıl doğrularsınız? Bu sır doğrudan AES anahtarı olarak kullanılmalı mıdır? ✓

Her iki tarafta da `pkeyutl -derive` ayrı ayrı çalıştırılıp çıktı dosyaları karşılaştırılır (`cmp`); ikisi de aynı sırra ulaşır. Hayır — ham DH çıktısı tekdüze rastgele değildir ve genelde tek bir değerdir; önce bir KDF'den (HKDF) geçirilip ihtiyaç kadar anahtar türetilmelidir.

? 22. `openssl verify` komutuna hem doğru hem alakasız bir ara sertifika verildiğinde ne olur? ✓

OpenSSL zinciri kurabildiği herhangi bir geçerli yoldan kurar (ör. doğrudan köke); alakasız sertifika yalnızca kullanılmayan bir aday olarak kalır, doğrulama yine de başarılı olabilir. Bu kriptografik değil operasyonel bir durumdur.

? 23. Bir ara CA sertifikasının `basicConstraints` alanı `CA:FALSE` ise ne olur? ✓

Bu sertifika artık başka sertifika imzalama yetkisine sahip değildir; onunla imzalanmış herhangi bir alt sertifikanın doğrulaması "invalid CA certificate" hatasıyla reddedilir.

? 24. `openssl ca -revoke` bir sertifikayı diskten siler mi? ✓

Hayır. Sertifika dosyası değişmez; yalnızca CA'nın `index.txt` defterine bir iptal kaydı eklenir. Bu kayıt daha sonra `-gencrl` ile imzalı bir CRL'e dönüştürülür.

? 25. `CKA_EXTRACTABLE=false` özneliği ne işe yarar ve ne zaman ayarlanmalıdır? ✓

Bir **HSM/PKCS#11** anahtarının düz metin olarak dışa aktarılmasını donanım düzeyinde engeller. Anahtar üretilirken (`C_GenerateKeyPair`) açıkça ayarlanmalıdır; sonradan değiştirilemez.

? 26. **ML-DSA-65** imzasının **Ed25519** imzasından kaç kat büyük olduğunu (bu haftaki ölçümde) belirtin ve pratik sonucunu açıklayın. ✓

Yaklaşık 52 kat büyük (3.309 bayt / 64 bayt). TLS el sıkışması, sertifika boyutu ve ağ trafiği artar; bu yüzden geçişte karma (hybrid) yaklaşımlar ve kriptto çevikliği önemlidir.

16. Kaynaklar ve ileri okuma

Ders kitabı — J. Viega, M. Messier, *Secure Programming Cookbook for C and C++*, O'Reilly, 2003:

- Tarif 5.2–5.4 (algoritma, anahtar uzunluğu ve kip seçimi), 5.11 (dolgu), 5.16 (hata yapmaya dirençli API)
- Tarif 6.3–6.4 (özet ve MAC seçimi), 6.10 (HMAC), 6.18 (şifreleme ve bütünlüğün birleştirilmesi), 6.19 (kendi MAC'ini yapma — uyarı), 6.21 (yeniden oynatmaya karşı MAC)
- Tarif 7.1–7.3 (açık anahtar kullanımı ve seçimi), 7.10–7.14 (RSA şifreleme, imza ve güvenli birleştirme), 7.16–7.17 (DER ve PEM)
- Tarif 8.15–8.19 (parola tabanlı anahtar değişimi, RSA ile kimlikli anahtar değişimi, Diffie–Hellman, DH + imza, PKI'siz doğrulama)
- Tarif 10.1, 10.4–10.5 (PKI ve X.509 doğrulama), 10.10–10.12 (CRL ve OCSP)

Açık standartlar ve kaynaklar

- NIST SP 800-57 Part 1 (anahtar yönetimi ve güvenlik düzeyleri), SP 800-131A (algoritma geçişleri), SP 800-38A/D (kipler, GCM), FIPS 186-5 (dijital imza), FIPS 203/204/205 (kuantum sonrası).
- RFC 2104 (HMAC), RFC 8017 ([PKCS#1](#) v2.2: OAEP, PSS), RFC 8032 (Ed25519), RFC 7748 (X25519), RFC 5280 (X.509 ve CRL), RFC 6960 (OCSP), RFC 6979 (deterministik ECDSA), RFC 8446 (TLS 1.3).
- OASIS [PKCS#11](#) v3; SoftHSM belgeleri; OpenSSL 3 kılavuz sayfaları (`genpkey`, `req`, `x509`, `verify`, `ca`, `pkeyutl`).
- MITRE CWE: CWE-327 (kırık algoritma), CWE-326 (yetersiz anahtar uzunluğu), CWE-295 (yanlış sertifika doğrulama), CWE-299 (iptal denetimi eksik), CWE-347 (imzanın yanlış doğrulanması), CWE-294 (yeniden oynatmayla kimlik atlatma).



Sözlük

Terim	Türkçe	Kısa tanım
Mode of operation	Çalışma kipi	Blok şifresini uzun mesajlara uygulama yöntemi
Padding oracle	Dolgu kâhini	Dolgu hatasını ayırt ederek şifreyi çözme saldırısı
Encrypt-then-MAC	Önce şifrele, sonra MAC	Önce şifreleyip şifreli metnin MAC'ini alma
Replay	Yeniden oynatma	Geçerli eski bir iletiyi tekrar gönderme
OAEP / PSS	—	RSA'nın güvenli şifreleme ve imza dolguları
Key agreement	Anahtar anlaşması	İki tarafın ortak sır üretmesi (DH, ECDH)
CA / RA	Sertifika / kayıt otoritesi	Sertifika çıkaran / kimlik doğrulayan taraf
SAN	Konu alternatif adı	Sertifikanın geçerli olduğu alan adları ve IP'ler
CRL / OCSP	İptal listesi / çevrimiçi durum protokolü	Sertifika iptalini bildirme yolları
Stapling	Yanıt ilişirme	Sunucunun OCSP yanıtını el sıkışmaya eklemesi
HSM	Donanım güvenlik modülü	Anahtarı dışa vermeyen kurcalamaya dayanıklı donanım
PKCS#11	—	Kriptografik belirteçlere erişim için standart API
Crypto agility	Kripto çevikliği	Algoritmaları kolayca değiştirebilme



Bir sonraki hafta

11. hafta — Whitebox kriptografi. Bu hafta anahtarı korumanın "donanıma emanet et" yolunu (HSM, [PKCS#11](#), `CKA_EXTRACTABLE=false`, §11) gördük. 11. hafta aynı soruyu donanım yokken sorar: bir anahtar, hiçbir HSM'e erişimi olmayan, tamamen yazılımdan ibaret bir uygulamanın içinde nasıl korunur? Tablo tabanlı whitebox AES ve ona karşı geliştirilen saldırılar (BGE, DCA, DFA), bu haftaki "anahtar hiç açığa çıkmasın" ilkesinin yazılım tarafındaki karşılığıdır.